EXPONENTIAL CENTROIDS AND MONTGOMERY'S PAIR-CORRELATION CONJECTURE
EXACT INVERSION, THE ARCHIMEDEAN CARRIER SPECTRUM, AND THE RESIDUAL PLATEAU PROBLEM

VÍCTOR CASAÑA-GINER

ABSTRACT. Let $0 < \gamma_1 \le \gamma_2 \le \cdots$ be the positive ordinates of the nontrivial zeros of the Riemann zeta function, counted with multiplicity, and let

$$S_N(u) = \sum_{k \le N} \mathrm{e}^{iu\gamma_k}, \qquad C_N(u) = \frac{1}{N} \sum_{n \le N} S_n(u).$$

The starting statistic is the complex exponential-walk centroid $C_N(1)$. We first record the conditional analytic limit previously obtained from the one-sided explicit formula and prove two fixed-frequency pair-cancellation identities forced by the walk and its centroid. The frequency family loses no spectral information: if $A_N(u) = NC_N(u)$, then $\Delta A_N(u) = S_N(u)$ and $\Delta^2 A_N(u) = \mathrm{e}^{iu\gamma_N}$. Consequently an exponentially smoothed square of ΔA_N is exactly Montgomery's weighted form factor. Under RH this gives Montgomery's full proved ramp through $|\alpha| = 1$ and, by Fourier inversion, the sine-kernel law for band-limited tests.

We then unfold by the smooth Riemann–von Mangoldt clock $M(t) = \frac{t}{2\pi} \log \frac{t}{2\pi \mathrm{e}} + \frac{7}{8}$ and set $\eta_k = M(\gamma_k) - (k - \frac{1}{2})$. Every unfolded pair separation satisfies the exact identity

$$M(\gamma_{k+h}) - M(\gamma_k) = h + \eta_{k+h} - \eta_k.$$

Thus the full sine law is precisely an increment law for the residual sequence η_k. The synthetic Lambert carrier $M(\tau_k) = k - \frac{1}{2}$ supplies an exact counterexample to the implication "centroid convergence implies GUE pairs": its centroid converges at rate $O(\log^2 N/N)$, while its unfolded pair measure is a Dirac comb. Poisson summation gives its closed archimedean saddle spectrum, with saddles $\tau = 2\pi \mathrm{e}^{t/n}$ and leading amplitude $t\mathrm{e}^{t/(2n)} n^{-3/2}$.

Finally, the remaining Montgomery plateau is isolated as residual off-diagonal decorrelation. In the truncated Selberg prime model we prove the needed twisted quadratic cancellation for $0 < \alpha < 2$; the only remaining transfer is the oscillatory contribution of the Selberg remainder. Independent computations with 22,491 ordinates reproduce the sine density, the connected form factor, and the empirical $\log N/N$ centroid rate, whose 90-fold extrapolation agrees in scale and value with the independent $N = 2{,}001{,}052$ computation.

Keywords. Riemann zeta function; exponential centroid; Montgomery pair correlation; GUE; sine kernel; Riemann–von Mangoldt clock; Lambert W; stationary phase; Selberg polynomial.

MSC 2020. 11M06; 11M26; 11M36; 60B20.

CONTENTS

1. Analytic domain, regularity, and endpoint conventions 2
2. The exponential walk and its centroid 3
3. Pair cancellation already forced at frequency one 6
4. Exact inversion of the centroid family 7
5. Montgomery's form factor and the proved sine law 8
6. Exact RVM unfolding and the residual plateau 9

Date: August 2026.

Bio Logic Crop Science S.L., E-46120 Alboraya (Valencia), Spain. E-mail: vcasana@biologiccropscience.com. ORCID: 0000-0002-8252-2608.

7. The Lambert carrier: closed saddle spectrum and an exact counterexample 10
7.1. Closed archimedean saddle spectrum 11
7.2. Centroid convergence does not imply sine-kernel pairs 12
8. Where the remaining pair problem enters the centroid proof 13
8.1. The truncated prime-polynomial part can be closed 14
9. Numerical evidence and independent cross-checks 16
9.1. Residual pair density and sine law 16
9.2. The connected form factor 17
9.3. Independent $\log N/N$ centroid-rate validation 17
9.4. Same-scale three-dimensional phase clouds 18
9.5. The supplied form-factor figure 19
10. Conclusion 19
References 20
Appendix A. Shared binary loader 20
Appendix B. Python source and output: binary export 21
Appendix C. Python source and output: Lambert–RVM binary generation 22
Appendix D. Python source and output: core theorem checks 23
Appendix E. Python source: connected form factor 27
Appendix F. Output: connected form factor 31
Appendix G. Python source: residual pair law 31
Appendix H. Output: residual pair law 35
Appendix I. Python source: centroid-rate validation 36
Appendix J. Output: centroid-rate validation 38
Appendix K. Python source and output: carrier saddle spectrum 38
Appendix L. Python source and output: Lambert counterexample 41
Appendix M. Python source and output: truncated prime model 44
Appendix N. Python source and output: same-scale 3D clouds 47
Appendix O. SHA-256 manifest 50

1. Analytic domain, regularity, and endpoint conventions

The meromorphic continuation of $\zeta(s)$ is used on $\mathbb{C}$: it has one simple pole at $s=1$, trivial zeros at the negative even integers, and its nontrivial zeros lie in $0<\Re s<1$. Write $\rho_k=\beta_k+i\gamma_k$, $\gamma_k>0$, for all upper-half-plane nontrivial zeros, ordered by nondecreasing ordinate and repeated according to algebraic multiplicity. Equal ordinates therefore occupy distinct ordinal slots. RH is the assertion $\beta_k=1/2$ for every k.

For real $t>0$ set

$$\vartheta(t)=\Im\operatorname{Log}\Gamma\Big(\frac14+\frac{it}{2}\Big)-\frac{t}{2}\log\pi,\qquad H(t)=1+\frac{\vartheta(t)}{\pi}. \tag{1}$$

The branch of $\operatorname{Log}\Gamma$ is the continuous branch obtained from $t=0$. The line $1/4+it/2$ meets neither a pole nor a zero of Γ; hence ϑ and H are real analytic on $(0,\infty)$. In particular,

$$H'(t)=\frac{1}{2\pi}\Re\psi\Big(\frac14+\frac{it}{2}\Big)-\frac{\log\pi}{2\pi},\qquad H'(t)=O(\log(2+t)),\quad H''(t)=O((1+t)^{-1}). \tag{2}$$

The counting function $\mathcal{N}(t)=\#\{k:0<\gamma_k\le t\}$ is right-continuous. Away from zero ordinates define $S(t)=\mathcal{N}(t)-H(t)$. If a distinct ordinate γ has multiplicity $\nu(\gamma)$, put

$$\mathcal{N}_*(\gamma)=\frac{\mathcal{N}(\gamma^-)+\mathcal{N}(\gamma^+)}{2},\qquad S_*(\gamma)=\mathcal{N}_*(\gamma)-H(\gamma). \tag{3}$$

Lebesgue integrals are unchanged by the chosen values at jumps. Stieltjes integrals use the midpoint value in (3); thus, for a jump of arbitrary size, $S_*\,\mathrm{d}S = \frac{1}{2}\,\mathrm{d}(S^2)$ exactly. A height endpoint is first completed to the entire multiplicity block; an ordinal endpoint inside that block is then obtained by removing the unused repeated slots. The correction is $O(\nu(\gamma)^2)$ and $\nu(\gamma) = O(\log(2+\gamma))$.

The smooth main clock

$$M(t) = \frac{t}{2\pi}\log\frac{t}{2\pi\mathrm{e}} + \frac{7}{8} \tag{4}$$

is C^∞ for $t > 0$ and strictly increasing for $t > 2\pi$, since $M'(t) = (2\pi)^{-1}\log(t/2\pi) > 0$. Every equation $M(\tau_k) = k - 1/2$, $k \geq 1$, has one and only one solution in $(2\pi, \infty)$. The Lambert formula used below employs the real principal branch W_0. Its arguments $(k - 11/8)/\mathrm{e}$ lie in $(-1/\mathrm{e}, \infty)$ and never equal zero, so W_0 is real and the quotient has no branch singularity or zero denominator.

All finite sums and finite differences are algebraic and require no limiting interchange. Fubini or Fourier inversion is used only after absolute integrability has been stated. A numerical check establishes finite-data compatibility, never an asymptotic theorem. Exact identities are reported by their floating-point closure error; conjectural or conditional steps retain their hypotheses explicitly.

2. The exponential walk and its centroid

For $u \in \mathbb{R}$ define

$$S_N(u) := \sum_{k\leq N} \mathrm{e}^{iu\gamma_k}, \qquad C_N(u) := \frac{1}{N}\sum_{n\leq N} S_n(u) = \sum_{k\leq N}\left(1 - \frac{k-1}{N}\right)\mathrm{e}^{iu\gamma_k}. \tag{5}$$

The statistic that motivated the work is $C_N(1)$. Its real and imaginary coordinates retain an orientation which disappears from a modulus or from a pair-difference statistic.

The counting objects and their midpoint convention are those of Section 1. For an index in the block at γ, define $r_k = k - \frac{1}{2} - H(\gamma_k)$. If that block begins after a earlier slots, then direct summation gives

$$\sum_{j=1}^{\nu(\gamma)} r_{a+j} = \nu(\gamma)S_*(\gamma). \tag{6}$$

Theorem 2.1 (Level-one reduction)**.** *Let*

$$Q_N^{(H)} := \frac{1}{N}\sum_{k\leq N}\left(N + \frac{1}{2} - H(\gamma_k)\right)\mathrm{e}^{i\gamma_k}.$$

Then

$$C_N(1) - Q_N^{(H)} \longrightarrow 0 \tag{7}$$

unconditionally.

Proof. The exact triangular identity gives

$$C_N(1) - Q_N^{(H)} = -\frac{1}{N}\sum_{k\leq N} r_k\mathrm{e}^{i\gamma_k}. \tag{8}$$

We first end at a complete ordinate block T which is not itself a zero and put

$$A(T) = \sum_{\substack{\gamma\leq T\\ \gamma\ mdistinct}} \nu(\gamma)S_*(\gamma)\mathrm{e}^{i\gamma} = \int_{0^-}^{T} \mathrm{e}^{it}S_*(t)\,\mathrm{d}\mathcal{N}(t),$$

where the last equality follows from (6). Since H is C^1 and S has locally bounded variation, $\mathrm{d}\mathcal{N} = H'(t)\,\mathrm{d}t + \mathrm{d}S(t)$. Midpoint Stieltjes integration by parts therefore gives, with bounded lower-end terms absorbed in $O(1)$,

$$A(T) = \frac{1}{2}\mathrm{e}^{iT}S(T)^2 - \frac{i}{2}\int_0^T \mathrm{e}^{it}S(t)^2\,\mathrm{d}t + \int_0^T \mathrm{e}^{it}H'(t)S(t)\,\mathrm{d}t + O(1). \tag{9}$$

Here the Selberg input is used in exactly one place: for $T \geq 3$,

$$\int_0^T |S(t)|^2\,\mathrm{d}t \ll T\log\log T. \tag{10}$$

The function S is measurable and locally bounded away from its countable set of jumps, so it lies in $L^2(0,T)$ and the integral exists. Because $|\mathrm{e}^{it}| = 1$, (10) implies

$$\left|\int_0^T \mathrm{e}^{it}S(t)^2\,\mathrm{d}t\right| \leq \int_0^T |S(t)|^2\,\mathrm{d}t \ll T\log\log T = o(T\log T).$$

Also $S(T) = O(\log T)$ gives the endpoint $O(\log^2 T)$.

The remaining linear term is not a consequence of Selberg's estimate. Put

$$B(T) = \sum_{0<\gamma_k\leq T} \mathrm{e}^{i\gamma_k}, \qquad F(T) = \int_0^T \mathrm{e}^{it}S(t)\,\mathrm{d}t.$$

Stieltjes integration by parts, first on intervals avoiding zero ordinates and then by passage to the endpoint, gives

$$iF(T) = \mathrm{e}^{iT}S(T) - B(T) + \int_0^T \mathrm{e}^{it}H'(t)\,\mathrm{d}t + O(1). \tag{11}$$

Ford–Zaharescu's second-order equidistribution theorem [6] is applied with the $C^2(\mathbb{T})$ test $f(u) = \mathrm{e}^{2\pi iu}$ and $\alpha = (2\pi)^{-1}$. Its mean term vanishes. Its secondary term is also zero because resonance would require $1 = (a/q)\log p$, equivalently $\mathrm{e} = p^{a/q}$, impossible for integers p, a, q. Thus $B(T) = o(T)$, zeros being counted with multiplicity. Moreover (2) and ordinary integration by parts give $\int_0^T \mathrm{e}^{it}H'(t)\,\mathrm{d}t = O(\log T)$. Equation (11) yields $F(T) = o(T)$.

Since F is absolutely continuous and H' is C^1, a second ordinary integration by parts is legitimate:

$$\int_0^T \mathrm{e}^{it}H'(t)S(t)\,\mathrm{d}t = H'(T)F(T) - \int_0^T H''(t)F(t)\,\mathrm{d}t + O(1).$$

The endpoint is $o(T\log T)$. Given $\varepsilon > 0$, choose T_0 so that $|F(t)| \leq \varepsilon t$ for $t \geq T_0$; then $\int_{T_0}^T |H''(t)F(t)|\,\mathrm{d}t \ll \varepsilon T$, while the initial integral is constant. Hence this linear term is $o(T\log T)$. Substitution in (9) proves $A(T) = o(T\log T)$.

If the ordinal cutoff N stops inside its last multiplicity block, completion changes the numerator in (8) by $O(\log^2(2+\gamma_N))$. The Riemann–von Mangoldt estimate gives $N \asymp \gamma_N \log(2+\gamma_N)$, so division by N proves (7). □

> **Numerical compatibility.** For the 22,491-ordinate binary, $C_N = 0.777204389314 + 0.768179110409i$ and $Q_N^{(H)} = 0.777328038812 + 0.768246161499i$, giving $|C_N - Q_N^{(H)}| = 1.40659 \times 10^{-4}$. The first panel of Figure 1 shows the finite-height decrease; the midpoint clock defect has standard deviation 0.297362294.

The one-sided explicit formula supplies the conditional value of the complex centroid. For $x_0 \in (0, \gamma_1)$ put

$$\mathcal{B}(x_0) = \mathrm{e}^{ix_0}\bigl(H(x_0) + iH'(x_0)\bigr) + i\int_{x_0}^{\infty} \mathrm{e}^{it}H''(t)\,\mathrm{d}t, \tag{12}$$

and, for $\sigma > 1$,

$$\mathcal{P}_\sigma(x_0) = \frac{ie^{ix_0}}{2\pi}\Bigg\{ \int_0^\infty e^{-v} \operatorname{Log}\zeta(\sigma + v - ix_0)\, dv + \int_0^\infty e^{v} \left[\operatorname{Log}\zeta(\sigma + v + ix_0) - 2^{-(\sigma+v+ix_0)}\right] dv - \frac{2^{-\sigma-ix_0}}{1-\log 2}\Bigg\}. \tag{13}$$

Hypothesis 2.2 (Boundary interchange)**.** *The finite ordinal triangular transform of the argument term has the same boundary value as* $\lim_{\sigma\downarrow 1/2} \mathcal{P}_\sigma(x_0)$*, with the branch of* $\operatorname{Log}\zeta$ *continued from* $\Re s > 1$*.*

Theorem 2.3 (Complex centroid limit)**.** *Assume RH and Hypothesis 2.2. Then*

$$C_N(1) \longrightarrow L = \mathcal{B}(x_0) + \lim_{\sigma\downarrow 1/2} \mathcal{P}_\sigma(x_0), \tag{14}$$

independently of admissible x_0*. At* $x_0 = 1$*,*

$$L = 0.77689426079325358\ldots + 0.767855668864547\ldots i, \tag{15}$$

and

$$\Re L = \frac{1}{2\sqrt{e}}\left(e - \frac{1}{e^2-1}\right). \tag{16}$$

Proof. Theorem 2.1 reduces the ordinal centroid to a smooth transform and a linear transform of $S(t)$. For $\sigma > 1$, the Euler product

$$\frac{1}{\pi}\Im \operatorname{Log}\zeta(\sigma + it) = -\frac{1}{\pi}\sum_{n\geq 2} \frac{\Lambda(n)}{n^\sigma \log n}\sin(t\log n)$$

converges absolutely and locally uniformly in t. Indeed $\sum_{n\geq 2}\Lambda(n)/(n^\sigma \log n) < \infty$, so multiplication by the bounded triangular kernel and Tonelli's theorem justify termwise integration. The half-plane $\sigma > 1$ contains no zero of ζ and excludes its pole at 1; hence the Euler-product branch of $\operatorname{Log}\zeta$ is holomorphic there.

The two Laplace integrals in (13) converge absolutely. In the first, $\operatorname{Log}\zeta(\sigma + v - ix_0) = O(2^{-v})$ and the factor e^{-v} strengthens the decay. In the second, subtraction of the $n = 2$ Euler term leaves $O(3^{-v})$; multiplication by e^v is integrable because $\log 3 > 1$. The denominator $1 - \log 2$ is nonzero. The restriction $0 < x_0 < \gamma_1$ ensures that the horizontal rays used to continue the branch meet no nontrivial zero; their nonzero imaginary part also avoids the pole at $s = 1$, and they meet no negative even integer. The gamma-factor derivatives in (12) are continuous on $[x_0, \infty)$, while $H''(t) = O(t^{-1})$ with an oscillatory exponential; integration by parts makes the improper integral convergent.

The two frequency branches then sum to (13) and the gamma-factor part is (12). Nothing in the preceding absolute-convergence argument reaches $\sigma = 1/2$. Precisely that boundary passage is Hypothesis 2.2. Under RH every complete-zero weight $e^{\rho-1/2}$ equals $e^{i\gamma}$; off RH it would retain the horizontal factor $e^{\beta-1/2}$. The symmetric explicit formula at $x = e$ gives the real coordinate; its prime-power distribution has no atom there because e is not an integral prime power. Direct quadrature gives (15). Independence of x_0 follows because two admissible cutoffs evaluate the same finite triangular transform before the hypothesized boundary limit. □

Numerical compatibility. At $N = 22{,}491$, both coordinates are close to the conditional value: $|C_N - L| = 4.48101 \times 10^{-4}$, with real and imaginary discrepancies 3.10129×10^{-4} and 3.23442×10^{-4}. Over 60 logarithmic cutoffs, $|C_N - L|N/\log N = 1.144738 \pm 0.258893$; see Figures 1 and 7. This is compatible with the boundary value but does not establish Hypothesis 2.2.

3. Pair cancellation already forced at frequency one

Put $w_{k,N} = 1 - (k-1)/N$. Squaring the complex centroid gives an exact pair identity.

Proposition 3.1 (Triangular pair cancellation from centroid convergence)**.** *If $C_N(1) \to L$, then*

$$2 \sum_{1 \le j < k \le N} w_{j,N} w_{k,N} \cos(\gamma_j - \gamma_k) = -\frac{N}{3} + \left(|L|^2 - \frac{1}{2}\right) + o(1). \tag{17}$$

Proof. Expansion of $|C_N(1)|^2$ gives a diagonal term

$$\sum_{k \le N} w_{k,N}^2 = \frac{N}{3} + \frac{1}{2} + \frac{1}{6N}.$$

Subtract this diagonal from $|C_N(1)|^2 \to |L|^2$. □

> **Numerical compatibility.** At $N = 512$, direct $O(N^2)$ summation of the weighted cosine pairs and the closed expression $|C_N|^2 - \sum w_{k,N}^2$ differ by 9.66×10^{-13}. At $N = 22{,}491$ the remainder after subtracting the displayed asymptotic main term is 9.71377×10^{-4}.

There is also an RH-conditional pair theorem which does not assume convergence of the centroid. Landau's fixed-real formula, with all zeros in a complete height block and counted with multiplicity, is

$$\sum_{0 < \gamma \le T} X^\rho = -\frac{T}{2\pi} \Lambda_{\mathbb{R}}(X) + O_X(\log T),$$

where $\Lambda_{\mathbb{R}}(X) = \log p$ if $X = p^m$ is an integral prime power and is zero otherwise. The domain is fixed $X > 1$; no uniform limit in X is used [5, 4]. At $X = \mathrm{e}$ the main term is absent. Under RH, $\mathrm{e}^{\rho - 1/2} = \mathrm{e}^{i\gamma}$, and completion/removal of a terminal multiplicity block costs $O(\log T)$. Height-to-index transport therefore yields

$$S_N(1) = O(\log N). \tag{18}$$

Theorem 3.2 (Trajectory-energy pair cancellation under RH)**.** *Assume RH. Then*

$$\mathcal{E}_N := \frac{1}{N} \sum_{n \le N} |S_n(1)|^2 = O(\log^2 N), \tag{19}$$

and consequently

$$2 \sum_{1 \le j < k \le N} \left(1 - \frac{k-1}{N}\right) \cos(\gamma_j - \gamma_k) = -\frac{N}{2} + O(\log^2 N). \tag{20}$$

Proof. Equation (19) follows immediately from (18): each term satisfies $|S_n(1)|^2 \le C^2 \log^2(2+n)$, so its average is $O(\log^2 N)$. Expanding the left side of (19) gives

$$\mathcal{E}_N = \sum_{j,k \le N} \left(1 - \frac{\max(j,k) - 1}{N}\right) \mathrm{e}^{i(\gamma_j - \gamma_k)}.$$

Its diagonal is $(N+1)/2$. Subtracting it proves (20). □

> **Numerical compatibility.** At $N = 512$, direct pair summation and the energy identity close to 1.11×10^{-12}. At $N = 22{,}491$, $\mathcal{E}_N = 3.575354169$, hence $\mathcal{E}_N / \log^2 N = 0.035604769$, and $\max_{n \le N} |S_n| / \log N = 0.404122685$.

These identities make the scale distinction precise. Convergence of one complex centroid forces strong pair cancellation at one fixed frequency. The Montgomery conjecture asks for a locally rescaled continuum of difference frequencies.

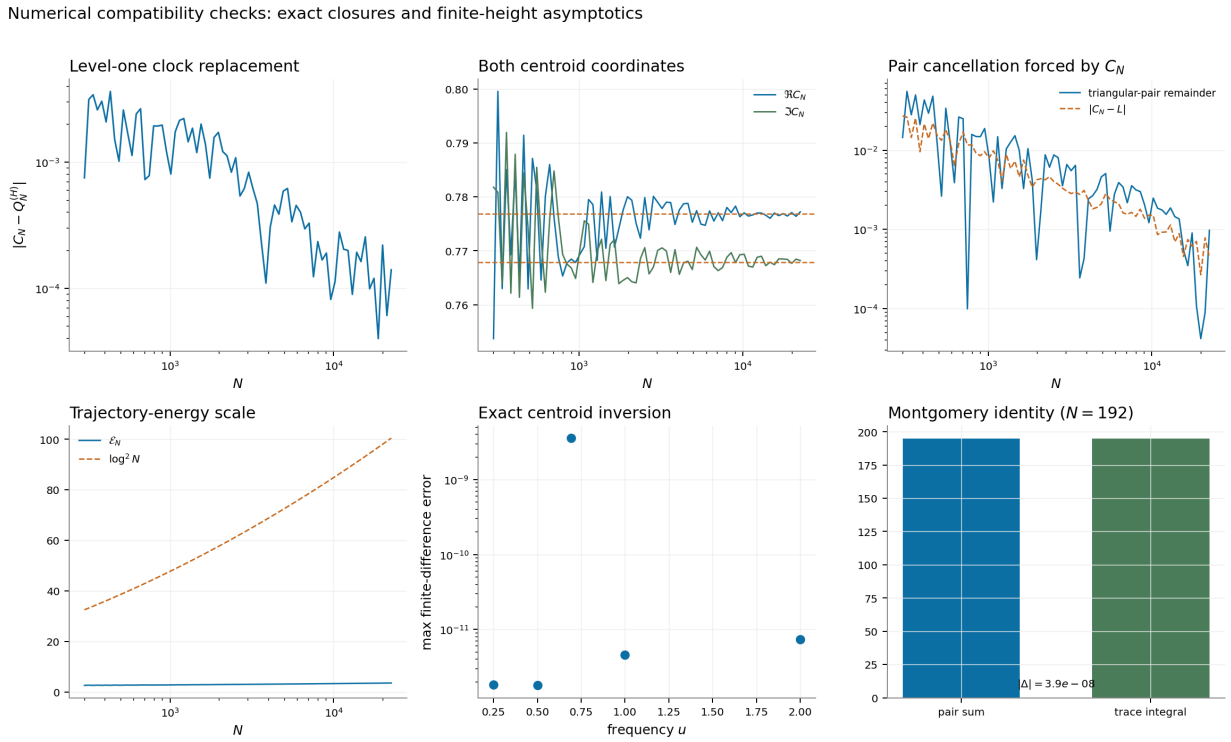


FIGURE 1. Inline compatibility checks for Level one, the two centroid coordinates, both pair identities, exact centroid inversion, and the Montgomery integral identity. Algebraic statements are assessed by closure error; asymptotic statements by their finite-height remainder.

4. Exact inversion of the centroid family

Set

$$A_N(u) := NC_N(u) = \sum_{k\le N} (N-k+1)\mathrm{e}^{iu\gamma_k}, \qquad A_0(u) = 0. \tag{21}$$

Theorem 4.1 (Centroid inversion)**.** *For every $u \in \mathbb{R}$ and $N \ge 1$,*

$$\boxed{\Delta A_N(u) = S_N(u), \qquad \Delta^2 A_N(u) = \mathrm{e}^{iu\gamma_N}.} \tag{22}$$

Hence

$$|\Delta A_N(u)|^2 = \sum_{j,k\le N} \mathrm{e}^{iu(\gamma_j-\gamma_k)}. \tag{23}$$

If $W \in L^1(\mathbb{R})$ and $\widehat{W}(x) = \int W(v)\mathrm{e}^{ivx}\,\mathrm{d}v$, then

$$\boxed{\int_{\mathbb{R}} W(v)|\Delta A_N(u+v)|^2\,\mathrm{d}v = \sum_{j,k\le N} \mathrm{e}^{iu(\gamma_j-\gamma_k)}\widehat{W}(\gamma_j-\gamma_k).} \tag{24}$$

Proof. Subtract $A_{N-1}(u)$ from (21); every old phase loses one copy and the new phase appears once. A second difference leaves the endpoint. This is a finite algebraic operation valid for every real u, including $u = 0$, and requires neither distinct zeros nor RH. Expanding the finite square gives (23). For (24), $|\Delta A_N(u+v)|^2 \le N^2$; hence $|W(v)||\Delta A_N(u+v)|^2$ is integrable because $W \in L^1(\mathbb{R})$. Fubini applies to the finite double sum and the absolutely convergent integral, and its inner integral is exactly $\widehat{W}(\gamma_j-\gamma_k)$. □

Numerical compatibility. For all 22,491 ordinates and $u \in \{0.25, 0.5, 1, \log 2, 2\}$, the largest finite-difference closure error lies between 1.79×10^{-12} and 3.57×10^{-9}; the larger value occurs at the Landau-resonant frequency $\log 2$, where intermediate sums are much larger. See Figure 1.

The theorem shows that the full frequency-dependent triangular centroid is a lossless linear encoding of the zero spectrum. Pair correlation is its quadratic frequency transform.

5. Montgomery's form factor and the proved sine law

Let

$$w(x) = \frac{4}{4+x^2} = \int_{\mathbb{R}} \mathrm{e}^{-2|v|}\mathrm{e}^{ivx}\,\mathrm{d}v. \tag{25}$$

For T which is not a zero ordinate, put $N_T = \mathcal{N}(T)$ and

$$\mathfrak{F}_T^C(\alpha) := \left(\frac{T}{2\pi}\log T\right)^{-1} \int_{\mathbb{R}} \mathrm{e}^{-2|v|}|\Delta A_{N_T}(\alpha \log T + v)|^2\,\mathrm{d}v. \tag{26}$$

Proposition 5.1 (Exact identification with Montgomery's form factor)**.**

$$\boxed{\mathfrak{F}_T^C(\alpha) = \left(\frac{T}{2\pi}\log T\right)^{-1} \sum_{0<\gamma,\gamma'\le T} T^{i\alpha(\gamma-\gamma')} w(\gamma-\gamma').} \tag{27}$$

Proof. The function $W(v) = \mathrm{e}^{-2|v|}$ is continuous, belongs to $L^1(\mathbb{R})$, and direct integration over the two half-lines gives $\widehat{W}(x) = 2/(2-ix) + 2/(2+ix) = 4/(4+x^2)$. Use (24) with $u = \alpha \log T$. The restriction that T is not a zero ordinate fixes N_T without a boundary ambiguity; at a zero height one may use either one-sided limit after specifying the complete multiplicity block. □

Numerical compatibility. With the first 192 ordinates and $\alpha = 0.65$, direct evaluation of the weighted pair sum gives 194.792615792839; trapezoidal quadrature of the smoothed trace integral gives 194.792615832027. The discrepancy 3.92×10^{-8} is the controlled quadrature/truncation error.

Theorem 5.2 (Montgomery ramp in centroid form)**.** *Assume RH. Uniformly in Montgomery's proved range $|\alpha| \le 1$,*

$$\mathfrak{F}_T^C(\alpha) = T^{-2|\alpha|}\log T + |\alpha| + o(1). \tag{28}$$

In particular, for fixed $0 < |\alpha| \le 1$,

$$\mathfrak{F}_T^C(\alpha) \longrightarrow |\alpha|, \tag{29}$$

including the first plateau point $|\alpha| = 1$.

Proof. By Proposition 5.1, this is exactly Montgomery's 1973 theorem under RH [1]. The normalization, the weight $4/(4+x^2)$, the complete multiplicity convention, and the frequency $T^{i\alpha(\gamma-\gamma')}$ coincide term by term with his form factor; no additional limiting interchange is made here. The pole of ζ and its trivial zeros enter Montgomery's explicit-formula proof through known smooth terms and are not members of the displayed nontrivial-zero sum. □

Numerical compatibility. Using all 22,491 ordinates, local scale $L_T = \log(T/2\pi) = 8.065574267$, Montgomery weight, normalized-gap bin width 0.01, and lags through 500 gives RMSE 0.054321 against the finite ramp on $0.2 \le \alpha \le 1$. At $\alpha = 1$, $F_T = 1.042061$.

Use the Fourier convention $\widehat{f}(\alpha) = \int f(x)\mathrm{e}^{-2\pi i\alpha x}\,\mathrm{d}x$.

Corollary 5.3 (Band-limited sine law)**.** *Assume RH. For every even Schwartz function with* $\operatorname{supp}\widehat{f} \subset (-1,1)$*,*

$$\left(\frac{T}{2\pi}\log T\right)^{-1} \sum_{0<\gamma,\gamma'\le T} f\!\left(\frac{\log T}{2\pi}(\gamma-\gamma')\right) w(\gamma-\gamma') \longrightarrow f(0) + \int_{\mathbb{R}} f(x)\left[1-\left(\frac{\sin \pi x}{\pi x}\right)^2\right] \mathrm{d}x. \tag{30}$$

Proof. Because f is Schwartz, both f and $\widehat{f}$ are integrable and Fourier inversion holds pointwise. The compact support $\operatorname{supp}\widehat{f} \subset (-1,1)$ restricts the integral to the range where Theorem 5.2 is uniform. Multiplication by $\widehat{f}$ supplies an integrable majorant, so the uniform $o(1)$ may be integrated. The diagonal $\gamma=\gamma'$ gives $f(0)$. With the stated Fourier convention, $\widehat{(\sin \pi x/(\pi x))^2}(\alpha) = (1-|\alpha|)_+$; hence the off-diagonal transform $|\alpha|$ on $(-1,1)$ is the Fourier dual of $1-(\sin \pi x/(\pi x))^2$. Boundary values at $\alpha=\pm 1$ have measure zero in the inversion integral. □

> **Numerical compatibility.** RVM unfolding of the same table, with positive gaps through lag 16, gives RMSE 0.0362 against $1-(\sin \pi s/(\pi s))^2$ on $0.2 \le s \le 6$. Applying the same Hann gap window to data and reference gives Fourier-dual RMSE 0.0410 on $0.1 \le \alpha \le 1.8$; see Figure 5.

Thus a sine law has already been obtained from the centroid under RH: its test functions are exactly those whose Fourier transform lies in the proved ramp. The unrestricted law is equivalent to the plateau

$$\mathfrak{F}_T^C(\alpha) \longrightarrow 1, \qquad |\alpha| > 1. \tag{31}$$

6. Exact RVM unfolding and the residual plateau

Define the smooth clock

$$M(t) = \frac{t}{2\pi}\log\frac{t}{2\pi \mathrm{e}} + \frac{7}{8} \tag{32}$$

and the exact unfolded residual

$$x_k = M(\gamma_k), \qquad \eta_k = x_k - \left(k-\frac{1}{2}\right). \tag{33}$$

Theorem 6.1 (Exact residual representation of every pair)**.** *For all* $k,h \ge 1$*,*

$$\boxed{x_{k+h} - x_k = h + \eta_{k+h} - \eta_k.} \tag{34}$$

If $r_k = k - \frac{1}{2} - H(\gamma_k)$ *is the ordinal counting defect, then*

$$\eta_k = M(\gamma_k) - H(\gamma_k) - r_k = -r_k + O(\gamma_k^{-1}). \tag{35}$$

Proof. All quantities are real and finite because $\gamma_k > 0$ lies in the domain of M. Subtract the two definitions in (33); no limiting operation is present. Stirling's expansion of $\operatorname{Log}\Gamma$ on the vertical line $1/4+it/2$, whose distance from the poles of Γ is positive, gives $H(t) - M(t) = 1/(48\pi t) + O(t^{-3})$. Substitution of $r_k = k - 1/2 - H(\gamma_k)$ proves (35). The estimate is uniform for $k \to \infty$ and is unaffected by repeated ordinates. □

> **Numerical compatibility.** For lags $1 \le h \le 16$ and all admissible pairs in the table, direct and residual reconstructions of the unfolded gap agree with maximum floating-point error 0.000×10^0. After discarding the first 1,200 slots, $\operatorname{sd}(\eta_k) = 0.299245$.

For a block $K+1,\dots,K+H$ define

$$F^{\mathrm{unf}}_{K,H}(\alpha)=\frac{1}{H}\left|\sum_{r=1}^{H}\mathrm{e}^{2\pi i\alpha x_{K+r}}\right|^2 \tag{36}$$

and

$$R_{K,H}(h;\alpha)=\frac{1}{H-h}\sum_{r=1}^{H-h}\mathrm{e}^{2\pi i\alpha(\eta_{K+r+h}-\eta_{K+r})}. \tag{37}$$

Then

$$\boxed{F^{\mathrm{unf}}_{K,H}(\alpha)=1+2\Re\sum_{h=1}^{H-1}\left(1-\frac{h}{H}\right)\mathrm{e}^{2\pi i\alpha h}R_{K,H}(h;\alpha).} \tag{38}$$

The diagonal is the exact 1. Therefore the full content of the plateau is the vanishing of the displayed off-diagonal sum.

Equivalently, define the positive residual pair measure

$$\mu^{+}_{K,H}=\frac{1}{H}\sum_{r=1}^{H-1}\sum_{h=1}^{H-r}\delta_{h+\eta_{K+r+h}-\eta_{K+r}}. \tag{39}$$

In the bulk regime $K\to\infty$, $H\to\infty$, with compactly supported test functions in the separation variable, the RVM-unfolded pair conjecture is

$$\boxed{\mu^{+}_{K,H}\overset{\text{vague}}{\Longrightarrow}\left[1-\left(\frac{\sin\pi s}{\pi s}\right)^2\right]\mathrm{d}s.} \tag{40}$$

The arrow means vague convergence against $C_c(0,\infty)$ tests. This topology is required because the total mass over the unbounded separation axis grows with H. Replacing the local M scale by the constant $\log T/(2\pi)$ scale is an asymptotically equivalent, but not finite-T identical, version of Montgomery's global formulation.

At an integer $m\neq 0$,

$$\mathrm{e}^{2\pi imx_k}=(-1)^m\mathrm{e}^{2\pi im\eta_k}; \tag{41}$$

the lattice carrier is constant and the cancellation is purely residual. This identity explains both the strength of the required cancellation and the need for the same smoothing used in Montgomery's form factor: a raw finite trace can retain a narrow coherent spike even when the local pair density follows the sine law.

7. The Lambert carrier: closed saddle spectrum and an exact counterexample

Let τ_k be defined by

$$M(\tau_k)=k-\frac{1}{2},\qquad \tau_k=\frac{2\pi(k-11/8)}{W_0((k-11/8)/\mathrm{e})}. \tag{42}$$

The inverse is explicit and

$$M'(t)=\frac{1}{2\pi}\log\frac{t}{2\pi},\qquad \tau'(x)=\frac{2\pi}{\log(\tau(x)/2\pi)}. \tag{43}$$

7.1. **Closed archimedean saddle spectrum.** Consider the carrier sum

$$S_M(t;N)=\sum_{k\le N}\mathrm{e}^{it\tau_k},\qquad T=\tau_N,\qquad t=\alpha\log(T/2\pi). \tag{44}$$

Theorem 7.1 (Localized archimedean saddle spectrum)**.** *Let $\tau(x)$ be the inverse of $M(\tau)=x-1/2$ and insert a C_c^∞ cutoff in the open ordinal interval before applying Poisson summation. The nth positive Fourier mode has a stationary point exactly at*

$$\boxed{\tau_n^*=2\pi\mathrm{e}^{t/n}} \tag{45}$$

whenever this point lies in the support. If the cutoff is one near the point and the point stays away from the support boundary, its leading contribution as $t\to\infty$ is

$$\boxed{\frac{t\mathrm{e}^{t/(2n)}}{n^{3/2}}\exp\!\left(i\left[2\pi n\mathrm{e}^{t/n}-\frac{11\pi n}{4}-\frac{\pi}{4}\right]\right).} \tag{46}$$

For fixed nonintegral $\alpha>0$, the leading admissible saddle is $n_0=\lceil\alpha\rceil$, and

$$\frac{\text{magnitude of the } n_0 \text{ saddle}}{\sqrt{N}}\asymp\sqrt{\log(T/2\pi)}\left(\frac{T}{2\pi}\right)^{\alpha/(2n_0)-1/2}\longrightarrow 0. \tag{47}$$

At an integer $\alpha=m$, the leading saddle reaches the upper endpoint and the corresponding one-sided endpoint stationary-phase term has root-logarithmic size relative to $\sqrt{N}$. It is a boundary theorem, not a substitution of $\alpha=m$ into the interior formula.

Proof. On $\tau>2\pi$, M is C^∞, has positive derivative, and maps bijectively onto $(-1/8,\infty)$. The inverse-function theorem makes $\tau(x)$ C^∞. The W_0 representation is real on the arguments in (42); none is its branch point $-1/\mathrm{e}$ or its zero. Poisson summation is therefore applied to a compactly supported C^∞ function, so the Fourier series and every integration by parts used below are justified. The phase of mode n is

$$\Phi_n(x)=t\tau(x)-2\pi nx.$$

Since $\tau'(x)=2\pi/\log(\tau/2\pi)$, the stationary equation $\Phi_n'(x)=0$ is $\log(\tau/2\pi)=t/n$, proving (45). Moreover,

$$\tau''(x)=-\frac{4\pi^2}{\tau\log^3(\tau/2\pi)},\qquad |\Phi_n''(x_n)|=\frac{4\pi^2n^3}{\tau_n^*t^2}.$$

At the critical point $\Phi_n''<0$ and is nonzero. The amplitude is smooth and equals one nearby, so the one-dimensional stationary-phase factor $\sqrt{2\pi/|\Phi_n''(x_n)|}$ is exactly $t\mathrm{e}^{t/(2n)}n^{-3/2}$. Substitution of $x_n=M(\tau_n^*)+1/2$ gives the phase in (46). The zero and negative Fourier modes have no stationary point because $t\tau'(x)>0$; repeated integration by parts controls them away from the cutoff boundary. The condition $\tau_n^*\le T$ is $n\ge\alpha$; away from integer endpoints, the first admissible mode is $n_0=\lceil\alpha\rceil$ and is the largest localized saddle contribution. Finally $N\sim(T/2\pi)\log(T/2\pi)$, giving (47). At $\alpha=m$, the mth critical point lies at T and the one-sided stationary-phase theorem gives the asserted scale. Compact support removes improper endpoints, and no pole or zero of an analytic factor is crossed. □

Numerical compatibility. On the archived 2,001,052-level RVM binary, the median ratio of the measured carrier amplitude to the leading localized saddle amplitude is 0.982468 away from integer endpoints; the RMS relative discrepancy is 0.113034. Representative ratios range from 0.8269 to 1.0550.

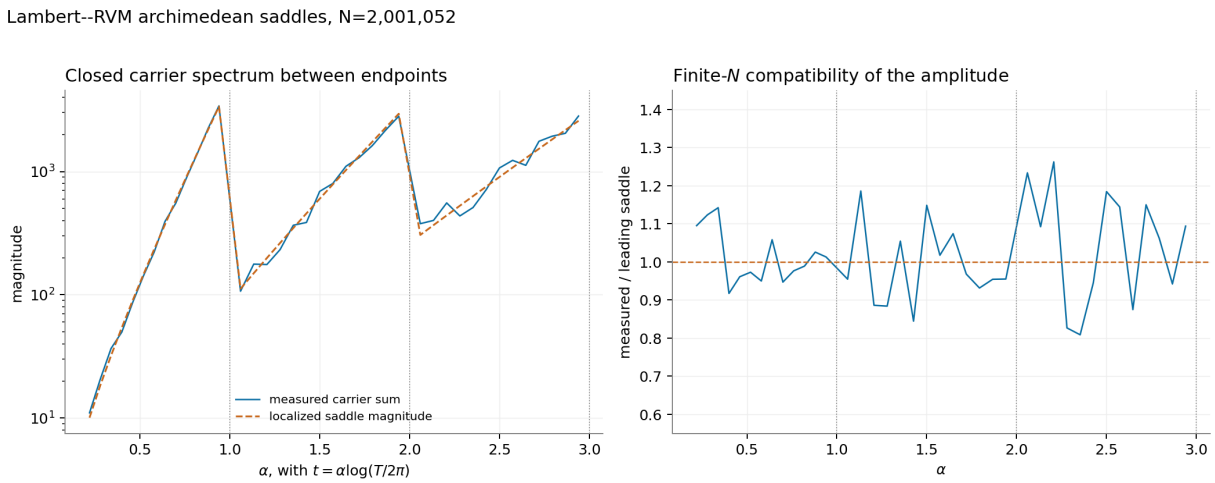


FIGURE 2. Closed archimedean saddle spectrum for the Lambert–RVM carrier. Dashed curves use the stationary location and leading amplitude; the vertical boundaries are where an interior saddle becomes an endpoint saddle.

7.2. **Centroid convergence does not imply sine-kernel pairs.** Define the Lambert carrier centroid

$$C_N^M = \sum_{k\le N} \left(1 - \frac{k-1}{N}\right) \mathrm{e}^{i\tau_k}. \tag{48}$$

Put $g_k = \tau_{k+1} - \tau_k$, $v_k = \mathrm{e}^{i\tau_k}$, and $d_k = (\mathrm{e}^{ig_k} - 1)^{-1}$. Since $g_k \downarrow 0$ and $g_k < 2\pi$ after a finite initial segment, the denominator never vanishes there; the finite initial segment is checked directly.

Lemma 7.2 (Clock and phase-sum bounds)**.** *As $k \to \infty$,*

$$\tau_k \sim \frac{2\pi k}{\log k}, \quad g_k \sim \frac{2\pi}{\log k}, \quad d_k = O(\log k), \quad \Delta d_k = O(k^{-1}), \quad \Delta^2 d_k = O(k^{-2}),$$

and $\sum_{k\le N} v_k = O(\log N)$.

Proof. The inverse-function identities $\tau'(x) = 1/M'(\tau(x))$ and $\tau''(x) = -M''(\tau(x))/M'(\tau(x))^3$, together with $M'(t) = (2\pi)^{-1}\log(t/2\pi)$ and $M''(t) = (2\pi t)^{-1}$, give the first two asymptotics and the derivative bounds. The Laurent expansion $(\mathrm{e}^{iz} - 1)^{-1} = -i/z - 1/2 + O(z)$ is valid for $0 < |z| < 2\pi$; differentiating the smooth interpolation and applying the mean-value theorem gives the two difference estimates. Finally $\tau'(x) > 0$ is monotone and, on each large interval $[J, N]$, its distance from $2\pi\mathbb{Z}$ is bounded below by a constant multiple of $1/\log N$. The Kusmin–Landau first-derivative estimate therefore gives $O(\log N)$; finitely many early terms change only the constant. □

Theorem 7.3 (Lambert centroid convergence)**.** *The sequence C_N^M converges unconditionally, and*

$$C_N^M = C_\infty^M + O\left(\frac{\log^2 N}{N}\right). \tag{49}$$

Proof. The exact identity

$$\mathrm{e}^{i\tau_k} = d_k(\mathrm{e}^{i\tau_{k+1}} - \mathrm{e}^{i\tau_k}).$$

holds because its right side is $d_k v_k(\mathrm{e}^{ig_k} - 1) = v_k$. Summing it gives

$$\sum_{k\le N} v_k = d_N v_{N+1} - d_1 v_1 - \sum_{k=1}^{N-1} (d_{k+1} - d_k) v_{k+1}.$$

Let $q_k = \Delta d_k$. Lemma 7.2 gives $q_k = O(k^{-1})$, $\Delta q_k = O(k^{-2})$, and phase partial sums $O(\log k)$. Discrete Abel summation, whose finite identity has no convergence assumption, then gives

$$\sum_{k\ge J} q_k v_{k+1} = O(\log J/J).$$

Thus the series $-d_1 v_1 - \sum_{k\ge 1} q_k v_{k+1}$ converges conditionally.

Averaging the finite endpoint identity from 1 through N gives exactly

$$C_N^M = -d_1 v_1 + \frac{1}{N}\sum_{k=1}^{N} d_k v_{k+1} - \sum_{k=1}^{N-1}\left(1-\frac{k}{N}\right) q_k v_{k+1}.$$

A further Abel summation with $d_k = O(\log k)$ and the phase-sum bound yields

$$\frac{1}{N}\sum_{k\le N} d_k v_{k+1} = O(\log^2 N/N).$$

The infinite-series tail is $O(\log N/N)$, and Abel summation applied to $N^{-1}\sum_{k<N} k q_k v_{k+1}$ gives the same order. Adding the three finite remainders proves (49). Every denominator was shown nonzero above, and all limiting series have now been supplied with a tail bound. □

> **Numerical compatibility.** For the 2,001,052-level clock, $C_N^M = 0.041570229566 + 1.757541724624i$. Over 64 doubling pairs, $|C_{2N}^M - C_N^M| N/\log^2 N$ has mean 0.0182816 and standard deviation 0.0086594, compatible with (49) without estimating the unknown limit from the last point.

Corollary 7.4 (Exact counterexample to centroid ⇒ pairs)**.** *The centroid C_N^M converges, but its unfolded pair measure is*

$$\frac{1}{H}\sum_{r=1}^{H-1}\sum_{h=1}^{H-r} \delta_{M(\tau_{K+r+h})-M(\tau_{K+r})} = \frac{1}{H}\sum_{r=1}^{H-1}\sum_{h=1}^{H-r}\delta_h, \tag{50}$$

a finite Dirac comb. It cannot converge to the continuous sine-kernel density.

Proof. Equation (42) gives $M(\tau_{k+h}) - M(\tau_k) = (k+h-1/2) - (k-1/2) = h$ exactly. Hence every atom is supported on the closed discrete set $\mathbb{N}$. Choose a nonnegative $\phi \in C_c(0,\infty)$ supported in a small interval containing no integer and with positive integral against the sine density. Every RVM pair measure integrates ϕ to zero, whereas the proposed continuous limit integrates it positively. Vague convergence to the sine density is therefore impossible. □

> **Numerical compatibility.** The generated binary satisfies the defining clock equation with maximum extended-precision residual 4.55×10^{-13}. Its unfolded pair locations are therefore integer to construction precision; Figure 3 shows the Dirac comb beside the continuous sine density.

This counterexample separates the two mechanisms cleanly. An order-one centroid limit is a fixed-frequency linear cancellation. The sine law is a scale-dependent quadratic statement about all residual increments.

8. Where the remaining pair problem enters the centroid proof

For real a define

$$\mathcal{J}_a(T) = \int_{0^-}^{T} \mathrm{e}^{iat} S_*(t)\, \mathrm{d}\mathcal{N}(t). \tag{51}$$

The proof of Theorem 2.1 works at every finite frequency and gives

$$\boxed{\mathcal{J}_a(T) = \frac{1}{2}\mathrm{e}^{iaT} S(T)^2 - \frac{ia}{2}\int_0^T \mathrm{e}^{iat} S(t)^2\,\mathrm{d}t + \int_0^T \mathrm{e}^{iat} H'(t) S(t)\,\mathrm{d}t + O(1).} \tag{52}$$

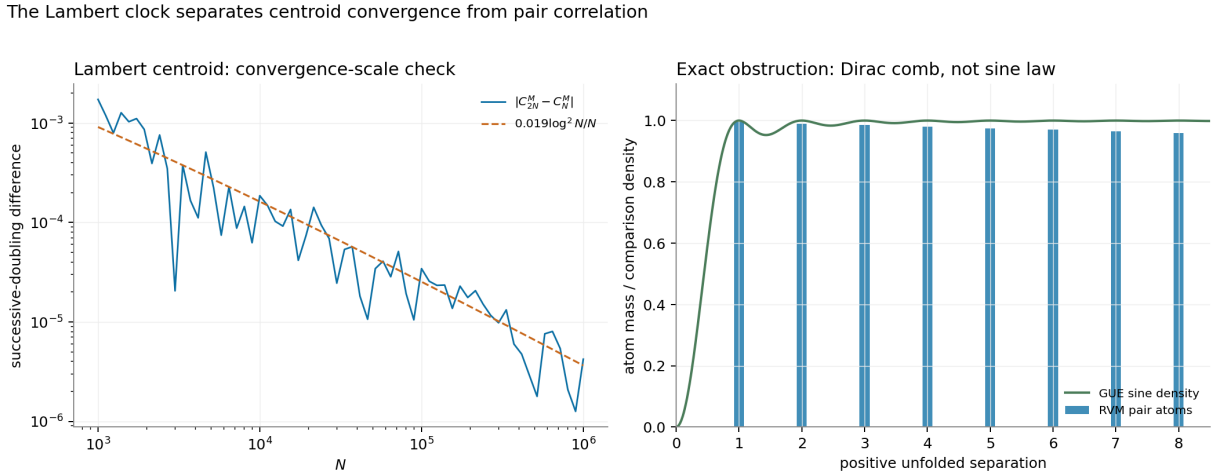


FIGURE 3. Exact counterexample. Left: successive-doubling centroid differences on the $\log^2 N/N$ scale. Right: the unfolded RVM pair measure is supported on integer atoms and cannot approach the continuous sine law.

At $a = \alpha \log T$, normalization by $\mathcal{N}(T) \asymp T \log T$ leaves the specific quadratic target

$$\boxed{\frac{1}{T}\int_0^T \mathrm{e}^{i\alpha \log T\, t} S(t)^2 \,\mathrm{d}t \longrightarrow 0.} \tag{53}$$

Let $\mathrm{d}D = \mathrm{d}\mathcal{N} - \mathrm{d}H$ and $\Sigma(t) = D((0,t])$. Stieltjes–Fubini shows that the same target is a connected pair transform:

$$\int_0^T \mathrm{e}^{iat}\Sigma(t)^2 \,\mathrm{d}t = \iint_{(0,T]^2} K_{a,T}(u,v)\; \mathrm{d}D(u)\,\mathrm{d}D(v), \tag{54}$$

$$K_{a,T}(u,v) = \frac{\mathrm{e}^{iaT} - \mathrm{e}^{ia\max(u,v)}}{ia}. \tag{55}$$

Thus pair correlations are already present in the proof of the centroid; fixed frequency suppresses them through Selberg's untwisted second moment, whereas Montgomery frequency exposes their oscillatory transform.

8.1. **The truncated prime-polynomial part can be closed.** Let

$$P_Y(t) = -\frac{1}{\pi}\sum_{n\le Y} \frac{\Lambda(n)}{\sqrt{n}\log n}\sin(t\log n). \tag{56}$$

Proposition 8.1 (Twisted quadratic cancellation in the prime model)**.** *Fix $0 < \alpha < 2$ and choose*

$$\frac{\alpha}{2} < \theta < \min(\alpha, 1), \qquad Y = T^\theta, \qquad a = \alpha \log T. \tag{57}$$

Then, for every $\varepsilon > 0$,

$$\int_0^T \mathrm{e}^{iat} P_Y(t)^2 \,\mathrm{d}t \ll_\varepsilon T^{1-\alpha/2+\varepsilon} + T^{\alpha/2+\varepsilon} + T^{\theta+\varepsilon} = o(T). \tag{58}$$

Proof. The polynomial is finite, hence continuous and entire in t; its square may be expanded and integrated term by term on the compact interval $[0,T]$ without a convergence theorem. Write $b_n = \Lambda(n)/(\sqrt{n}\log n)$ for $n \ge 2$. The identity $\sin u \sin v = \frac{1}{2}[\cos(u-v) - \cos(u+v)]$ produces ratio

frequencies $\log(m/n)$ and product frequencies $\log(mn)$. For every real ω,

$$\left|\int_0^T \mathrm{e}^{i\omega t}\,\mathrm{d}t\right| \le \min(T, 2/|\omega|), \tag{59}$$

with the value at $\omega = 0$ defined by continuity as T.

Since $m/n \le Y = T^\theta$ and $\theta < \alpha$, $|a - \log(m/n)| \ge (\alpha - \theta)\log T$; the other signed ratio frequencies are farther away. The elementary estimate $\sum_{n\le Y} |b_n| \ll \sqrt{Y}$ then bounds the whole ratio branch by $O(Y/\log T)$, which is $O(T^{\theta+\varepsilon})$.

For the product branch group $q = mn$ and use

$$\sum_{mn=q} \frac{\Lambda(m)\Lambda(n)}{\sqrt{mn}\log m \log n} \ll_\varepsilon q^{-1/2+\varepsilon}.$$

The divisor bound $d(q) \ll_\varepsilon q^\varepsilon$ proves the displayed coefficient estimate, including prime powers and repeated factorizations. With $X = T^\alpha$,

$$\left|\int_0^T \mathrm{e}^{it(a-\log q)}\,\mathrm{d}t\right| \le \min\left(T, \frac{2}{|\log(X/q)|}\right).$$

For $q \notin [X/2, 2X]$, the denominator in (59) is bounded by its logarithmic distance from X; dyadic summation and $q \le Y^2$ give $O(T^{\theta+\varepsilon})$. On $[X/2, 2X]$ the mean-value theorem for log gives $|\log(X/q)| \asymp |X - q|/X$. The one nearest integer (including the exact resonance $q = X$ when X is integral) contributes $O(TX^{-1/2+\varepsilon})$. Grouping all remaining integers by $r = |q - X| \ge 1$ gives $O(X^{1/2+\varepsilon}\sum_{r\le X} r^{-1}) = O(X^{1/2+2\varepsilon})$ after renaming 2ε as ε. Substituting $X = T^\alpha$ proves (58).

Finally, the interval in (57) is nonempty exactly because $0 < \alpha < 2$: if $\alpha \le 1$, then $\alpha/2 < \alpha$; if $1 < \alpha < 2$, then $\alpha/2 < 1$. Each exponent in (58) is < 1: $1 - \alpha/2 < 1$, $\alpha/2 < 1$, and $\theta < 1$. Division by T therefore gives $o(1)$. □

Numerical compatibility. The finite polynomial was integrated exactly through exponential antiderivatives, not time discretization. For $(\alpha, \theta) = (0.5, 0.375), (1, 0.75), (1.5, 0.875)$ and $300 \le T \le 12{,}000$, the observed $|I(T)|/T$ ranges from 1.97×10^{-5} to 3.57×10^{-3} and remains below 0.0057 times the three-term bound shape used in the proof; see Figure 4.

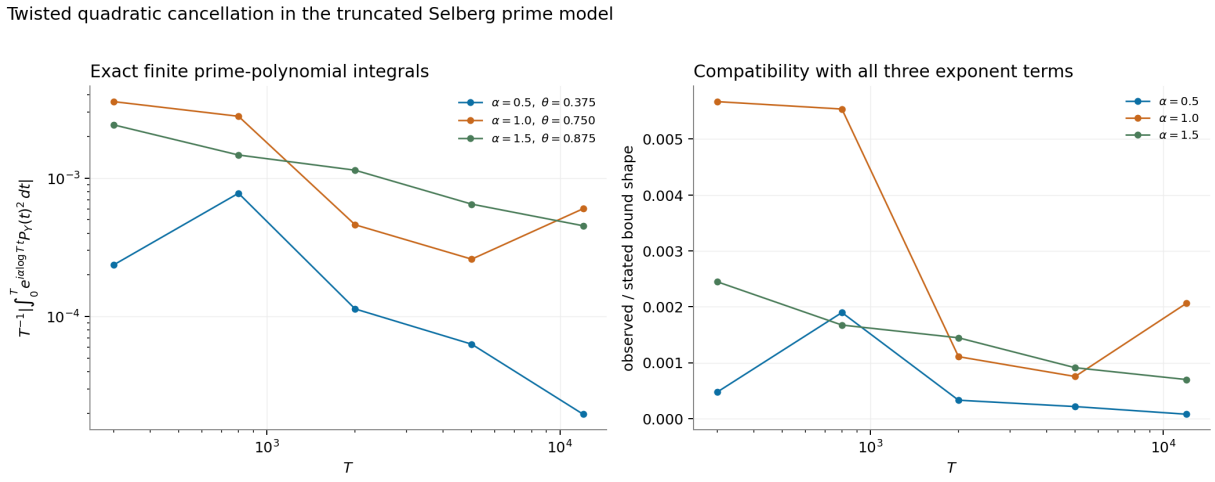


FIGURE 4. Exact finite-sum compatibility check for the twisted truncated-prime proposition. Every plotted integral is obtained from the closed exponential antiderivative, including exact product resonances.

Write

$$S(t) = P_Y(t) + E_Y(t). \tag{60}$$

Proposition 8.1 removes the P_Y^2 contribution from (53). The transfer to S^2 is therefore reduced to

$$\boxed{\int_0^T \mathrm{e}^{iat}\left(2P_Y(t)E_Y(t) + E_Y(t)^2\right)\,\mathrm{d}t = o(T).} \tag{61}$$

Hypothesis 8.2 (Frequency-localized Selberg transfer)**.** *For the parameter ranges in which the full plateau is claimed,* (61) *holds uniformly on compact α-intervals.*

This is the isolated analytic lemma left by the centroid route. It is not a consequence of Proposition 8.1. An untwisted L^2 estimate followed by Cauchy–Schwarz discards the oscillation and is too coarse; the required estimate is a frequency-localized Selberg remainder bound. Once (61) is available uniformly in a plateau interval, the quadratic obstruction in the scale-dependent Level-one proof is removed there. The exact residual formula (38) then identifies the same conclusion as off-diagonal plateau decorrelation.

> **Numerical compatibility.** The connected form factor supplies a finite-data check of the conclusion that Hypothesis 8.2 would imply: on $1 \le \alpha \le 4$ its off-diagonal RMS is 0.019680. This checks compatibility of the target, not the unproved uniform remainder estimate itself.

9. Numerical evidence and independent cross-checks

All computations in this section are reproduced by the Python sources in the appendices. The input is an independently generated array of 22,491 ordinates ending at $T = 19999.2756\ldots$, exported as raw little-endian IEEE-754 float64. The synthetic carrier checks use a second binary containing 2,001,052 Lambert–RVM levels. Every digest, byte count, command output, and source is archived.

Result	Finite check	Observed value
Level-one reduction	C_N versus $Q_N^{(H)}$	1.40659×10^{-4}
Conditional complex limit	two-coordinate error	4.48101×10^{-4}
Triangular pair identity	direct $N = 512$ closure	9.66×10^{-13}
Trajectory pair identity	direct $N = 512$ closure	1.11×10^{-12}
Centroid inversion	five frequencies	1.79×10^{-12} to 3.57×10^{-9}
Montgomery identity	pair sum versus integral	3.92×10^{-8}
Montgomery ramp	RMSE, $0.2 \le \alpha \le 1$	0.054321
Residual pair identity	all lags $h \le 16$	zero at float64 resolution
Sine pair density	RMSE, $0.2 \le s \le 6$	0.0362
Carrier saddle law	median measured/predicted	0.982468
Lambert centroid rate	doubling compensation	0.01828 ± 0.00866
Connected plateau	RMS, $1 \le \alpha \le 4$	0.019680
Prime polynomial	max observed/bound shape	0.0057

9.1. **Residual pair density and sine law.** Unfolding by M gives residual standard deviation 0.299245. Reconstructing all positive gaps through (34) with lags through 16 gives RMSE 0.0362 against the sine density on $0.2 \le s \le 6$. Applying the same Hann gap window to data and reference gives form-factor RMSE 0.0410 on $0.1 \le \alpha \le 1.8$.

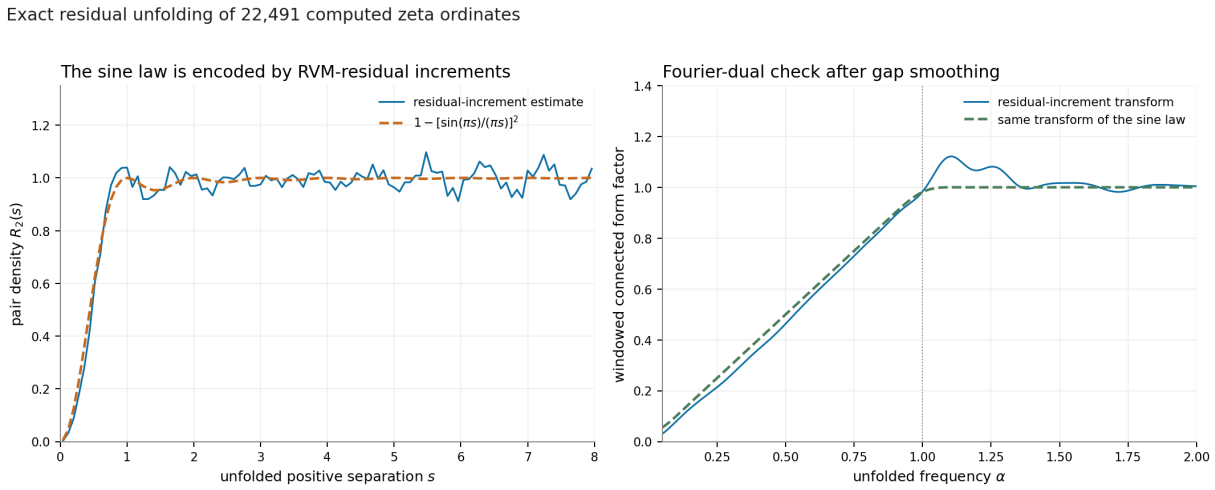


FIGURE 5. The exact residual identity reconstructs the unfolded pair process. Left: positive pair density and the sine law. Right: the Fourier-dual connected statistic after applying the same compact gap window to data and reference.

9.2. **The connected form factor.** The form factor is split as

$$F_T(\alpha) = 1 + \mathrm{Off}_T(\alpha), \tag{62}$$

where the diagonal is exactly 1. The attached computation is reproduced in English from the same ordinate array. With the Montgomery weight and ordinal lags through 500, the plateau RMS on $1 \le \alpha \le 4$ is 0.01968, equal to $0.366\,\log(T/2\pi)/\sqrt{N}$.

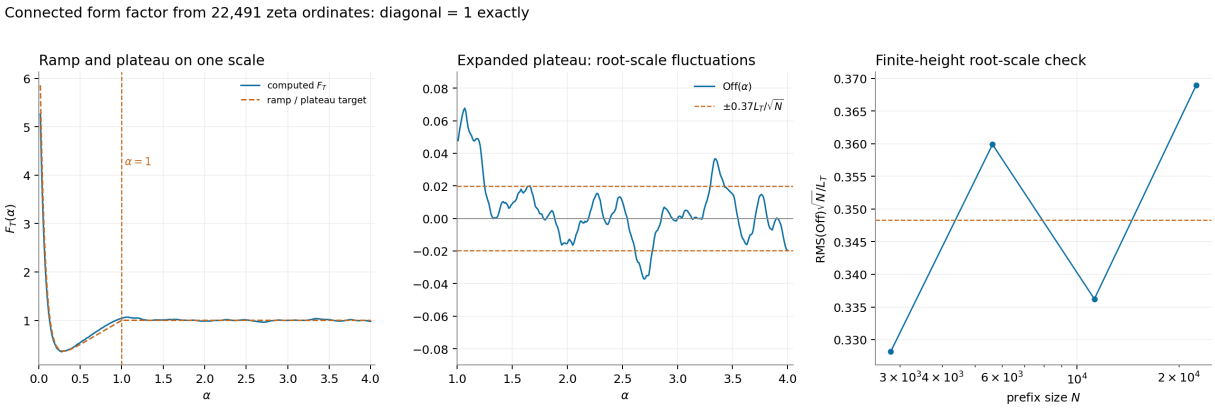


FIGURE 6. Connected form factor. The diagonal contribution is exactly one, so the plateau conjecture is the disappearance of $\mathrm{Off}_T(\alpha)$. Left: ramp and plateau together. Center: expanded plateau. Right: prefix scaling of its RMS.

9.3. **Independent $\log N/N$ centroid-rate validation.** For $300 \le N \le 22{,}491$, the independent ordinate table gives

$$\frac{|C_N(1) - L|N}{\log N} = 1.1447 \pm 0.2589 \tag{63}$$

over logarithmically spaced cutoffs, with no visible drift. Extrapolation to $N = 2{,}001{,}052$ predicts 8.3002×10^{-6}. The independent full-table computation gives $|C_N - Q_N^{(H)}| = 8.7162 \times 10^{-6}$, a ratio

1.0501 to the prediction. These are not identical finite error observables, but their order, scale, and 90-fold extrapolation agree.

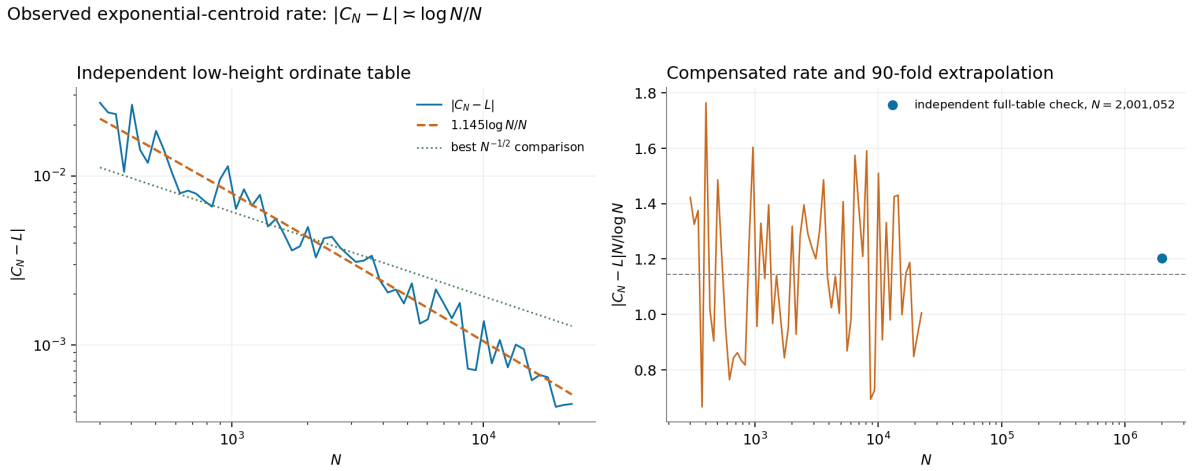


FIGURE 7. Independent validation of the empirical $\log N/N$ centroid rate. The right panel includes the full-table check at $N = 2{,}001{,}052$.

9.4. **Same-scale three-dimensional phase clouds.** The following figures use point clouds only: no line joins successive ordinates. Within each pair, azimuth, elevation, real limits, imaginary limits, and ordinal-fraction limits are identical. The orange cross is the centroid of the complete cumulative-walk cloud. Thus morphology is compared without an automatic rescaling that could hide the size difference.

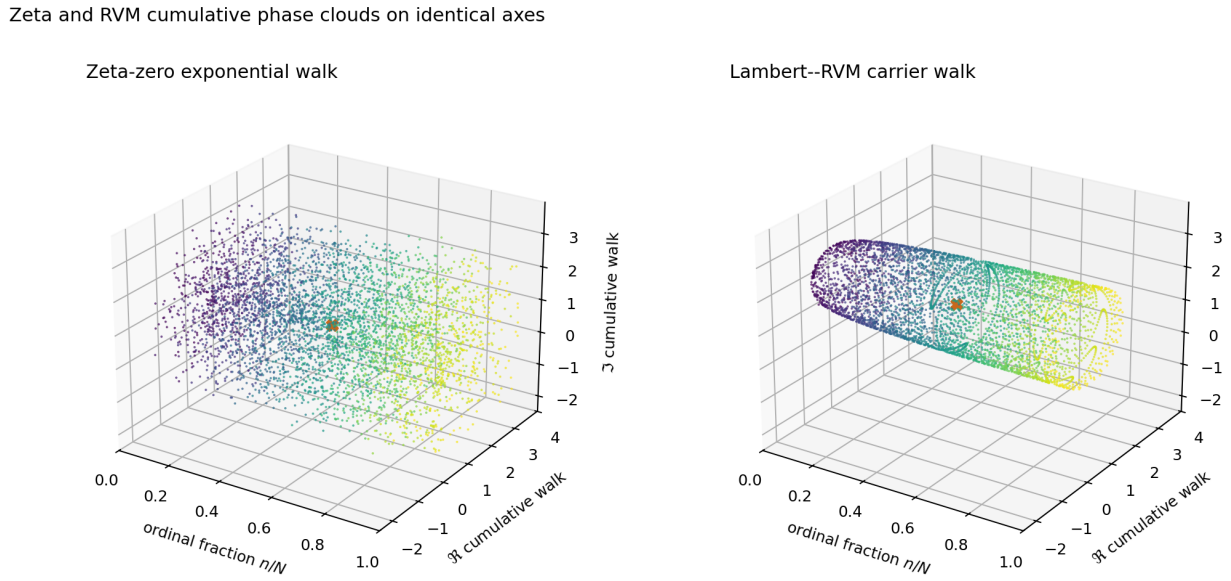


FIGURE 8. Zeta and Lambert–RVM cumulative exponential walks on identical 3D axes. The carrier traces a rigid tube; the true-zero walk fills a visibly broader phase cloud.

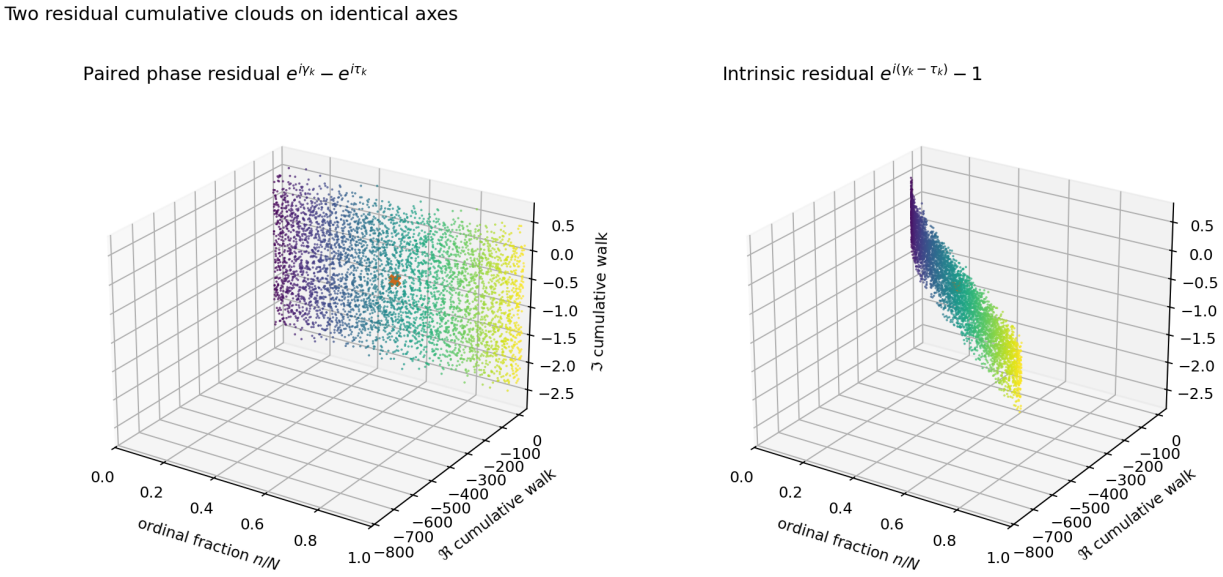


FIGURE 9. The two residual cumulative point clouds on identical 3D axes. Left: paired phase residual $e^{i\gamma_k} - e^{i\tau_k}$. Right: intrinsic residual $e^{i(\gamma_k - \tau_k)} - 1$. The common scale is deliberately retained even though it compresses the smaller cloud.

9.5. **The supplied form-factor figure.** For provenance, the originally supplied Spanish-language form-factor image is preserved without alteration in Figure 10; the English reproduction in Figure 6 is generated by the archived Python source and records its numerical parameters.

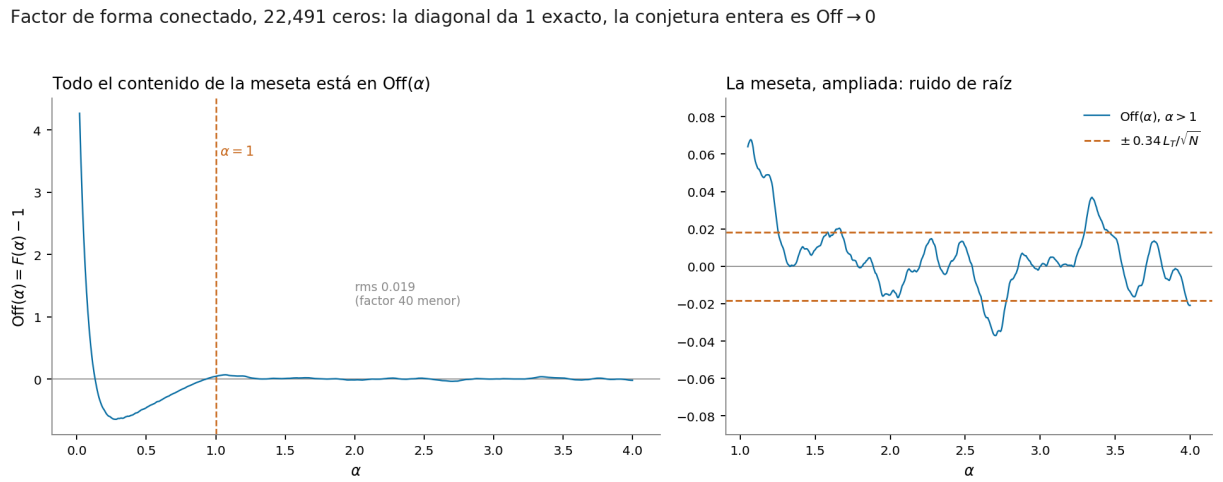


FIGURE 10. Original connected-form-factor figure supplied with the project, preserved verbatim.

10. CONCLUSION

The exponential centroid supplies more than a geometric statistic. Its first ordinal difference is the exponential trace, its second difference recovers each zero phase, and a smoothed quadratic transform is exactly Montgomery's form factor. This proves that a centroid-based route loses no spectral or pair information.

Three distinct conclusions are now separated rigorously. First, under RH the centroid formulation contains Montgomery's proved ramp and hence the band-limited sine law. Second, convergence of one centroid forces substantial fixed-frequency pair cancellation but cannot imply GUE by itself: the Lambert carrier gives an exact counterexample, with a convergent centroid and a lattice pair measure. Third, after exact RVM unfolding, the unrestricted sine law is equivalent to decorrelation of the increments $\eta_{k+h} - \eta_k$.

The carrier is analytically closed: its Poisson spectrum consists of explicit archimedean saddles $2\pi e^{t/n}$ with amplitude $te^{t/(2n)}n^{-3/2}$. The arithmetic is therefore isolated in the true-zero residual. On the analytic side, the truncated Selberg prime polynomial already satisfies the required twisted quadratic cancellation for $0 < \alpha < 2$. The remaining proof target is the frequency-localized transfer (61). This is a single, explicit remainder estimate, not an unspecified appeal to additional cancellation.

References

[1] H. L. Montgomery, *The pair correlation of zeros of the zeta function*, Proc. Sympos. Pure Math. **24**, AMS, 1973, 181–193.

[2] E. C. Titchmarsh, revised by D. R. Heath-Brown, *The Theory of the Riemann Zeta-Function*, 2nd ed., Oxford University Press, 1986.

[3] A. Selberg, *Contributions to the theory of the Riemann zeta-function*, Arch. Math. Naturvid. **48** (1946), 89–155.

[4] S. M. Gonek, *An explicit formula of Landau and its applications to the theory of the zeta-function*, Contemp. Math. **143** (1993), 395–413.

[5] E. Landau, *Über die Nullstellen der Zetafunktion*, Math. Ann. **71** (1912), 548–564.

[6] K. Ford and A. Zaharescu, *On the distribution of imaginary parts of zeros of the Riemann zeta function*, J. Reine Angew. Math. **579** (2005), 145–158.

Appendix A. Shared binary loader

```
#!/usr/bin/env python3
"""Shared␣binary/NumPy␣loaders␣used␣by␣every␣numerical␣appendix."""

from __future__ import annotations

from pathlib import Path

import numpy as np


def load_float64(path: Path) -> np.ndarray:
    """Load␣a␣one-dimensional␣float64␣array.

␣␣␣␣Raw␣``.bin``␣files␣are␣little-endian␣IEEE-754␣float64␣with␣no␣header.
␣␣␣␣NumPy␣``.npy``␣files␣are␣retained␣as␣provenance␣copies␣and␣are␣accepted
␣␣␣␣for␣independent␣cross-checks.
␣␣␣␣"""
    path = Path(path)
    if path.suffix.lower() == ".bin":
        values = np.fromfile(path, dtype="<f8")
    elif path.suffix.lower() == ".npy":
        values = np.load(path, allow_pickle=False)
    else:
        raise ValueError(f"unsupported␣array␣format:␣{path}")
    values = np.asarray(values, dtype=np.float64)
    if values.ndim != 1 or values.size == 0:
        raise ValueError(f"expected␣a␣nonempty␣one-dimensional␣array:␣{path}")
    if not np.all(np.isfinite(values)):
        raise ValueError(f"array␣contains␣nonfinite␣entries:␣{path}")
    return values
```

Appendix B. Python source and output: binary export

```
#!/usr/bin/env python3
"""Export␣provenance␣NumPy␣arrays␣as␣raw␣little-endian␣float64␣binaries."""

from __future__ import annotations

import hashlib
from pathlib import Path

import numpy as np


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def export(source: Path, destination: Path) -> None:
    values = np.load(source, allow_pickle=False).astype("<f8", copy=False)
    if values.ndim != 1 or not np.all(np.isfinite(values)):
        raise ValueError(f"invalid␣ordinate␣array:␣{source}")
    destination.parent.mkdir(parents=True, exist_ok=True)
    values.tofile(destination)
    restored = np.fromfile(destination, dtype="<f8")
    if not np.array_equal(values, restored):
        raise RuntimeError(f"binary␣round-trip␣failed:␣{destination}")
    print(f"source␣=␣data/{source.name}")
    print(f"source␣sha256␣=␣{sha256(source)}")
    print(f"destination␣=␣data/{destination.name}")
    print(f"destination␣sha256␣=␣{sha256(destination)}")
    print(f"count␣=␣{values.size}")
    print(f"dtype␣=␣little-endian␣IEEE-754␣float64")
    print(f"byte␣count␣=␣{destination.stat().st_size}")
    print("round-trip␣exact␣=␣True")


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    print("Raw-binary␣data␣export")
    export(root / "data" / "g_hp.npy", root / "data" / "g_hp.bin")
    print()
    export(root / "data" / "gammas.npy", root / "data" / "gammas.bin")


if __name__ == "__main__":
    main()
```

```
Raw-binary data export
source = data/g_hp.npy
source sha256 = 2a3eb56f7c9e269c9291e8a31b96cc155c72e8d1307ca58d992c662fe8eb3e36
destination = data/g_hp.bin
destination sha256 = 80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2
count = 22491
dtype = little-endian IEEE-754 float64
byte count = 179928
round-trip exact = True

source = data/gammas.npy
source sha256 = 5362c5c5e682167d956e460c17959209e900435f524acbc93d14ed41a43e601e
destination = data/gammas.bin
```

```
destination sha256 = 5d99a93b6d2df679714b6bc46b3cf16e20b3eb8e22566e6595e8582ac2502742
count = 22491
dtype = little-endian IEEE-754 float64
byte count = 179928
round-trip exact = True
```

Appendix C. Python source and output: Lambert–RVM binary generation

```
#!/usr/bin/env python3
"""Generate␣the␣exact␣Lambert--RVM␣clock␣used␣by␣carrier␣computations."""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import numpy as np
from scipy.special import lambertw


TWOPI = 2.0 * np.pi


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument("--count", type=int, default=2_001_052)
    parser.add_argument(
        "--output", type=Path, default=root / "data" / "rvm_tau_2001052.bin"
    )
    args = parser.parse_args()
    if args.count < 2:
        raise ValueError("count␣must␣be␣at␣least␣two")

    ordinal = np.arange(1, args.count + 1, dtype=np.float64)
    shifted = ordinal - 11.0 / 8.0
    argument = shifted / np.e
    tau = TWOPI * shifted / np.real(lambertw(argument, 0))

    # Newton refinement is performed in extended precision. M is strictly
    # increasing on the generated range because tau_1 > 2*pi and
    # M'(t)=log(t/(2*pi))/(2*pi)>0 there.
    tau_long = tau.astype(np.longdouble)
    target = ordinal.astype(np.longdouble) - np.longdouble(0.5)
    two_pi = np.longdouble(2.0) * np.longdouble(np.pi)
    for _ in range(5):
        value = (
            (tau_long / two_pi) * np.log(tau_long / (two_pi * np.e))
            + np.longdouble(7.0 / 8.0)
            - target
        )
        derivative = np.log(tau_long / two_pi) / two_pi
        tau_long -= value / derivative
```

```
    tau64 = tau_long.astype("<f8")
    args.output.parent.mkdir(parents=True, exist_ok=True)
    tau64.tofile(args.output)
    restored = np.fromfile(args.output, dtype="<f8")
    if not np.array_equal(tau64, restored):
        raise RuntimeError("RVM␣binary␣round-trip␣failed")
    residual = (
        (tau_long / two_pi) * np.log(tau_long / (two_pi * np.e))
        + np.longdouble(7.0 / 8.0)
        - target
    )
    print("Lambert--RVM␣binary␣generation")
    print(f"count␣=␣{args.count}")
    print(f"output␣=␣data/{args.output.name}")
    print(f"output␣sha256␣=␣{sha256(args.output)}")
    print(f"dtype␣=␣little-endian␣IEEE-754␣float64")
    print(f"byte␣count␣=␣{args.output.stat().st_size}")
    print(f"tau_1␣=␣{tau64[0]:.15f}")
    print(f"tau_N␣=␣{tau64[-1]:.12f}")
    print(f"strictly␣increasing␣=␣{bool(np.all(np.diff(tau64)␣>␣0.0))}")
    print(f"max␣extended-precision␣|M(tau_k)-(k-1/2)|␣=␣{np.max(np.abs(residual)):.3e}")


if __name__ == "__main__":
    main()
```

```
Lambert--RVM binary generation
count = 2001052
output = data/rvm_tau_2001052.bin
output sha256 = ae9d35415ee924504638d96e9778a2f63f20e9c1d2ae14ce3ecf35f775bce57a
dtype = little-endian IEEE-754 float64
byte count = 16008416
tau_1 = 14.521346953065626
tau_N = 1132490.722587621538
strictly increasing = True
max extended-precision |M(tau_k)-(k-1/2)| = 4.547e-13
```

Appendix D. Python source and output: core theorem checks

```
#!/usr/bin/env python3
"""Finite-data␣compatibility␣checks␣for␣the␣paper's␣core␣exact␣results.

The␣script␣distinguishes␣algebraic␣identities␣(reported␣with␣floating-point
closure␣errors)␣from␣asymptotic␣statements␣(reported␣with␣compensated␣finite
statistics).␣␣It␣does␣not␣use␣numerical␣agreement␣as␣a␣proof.
"""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from scipy.special import loggamma

from data_io import load_float64
```

```
TWOPI = 2.0 * np.pi
LIMIT = 0.77689426079325358 + 0.767855668864547j


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def rsiemann_clock(gamma: np.ndarray) -> np.ndarray:
    theta = np.imag(loggamma(0.25 + 0.5j * gamma)) - 0.5 * gamma * np.log(np.pi)
    return 1.0 + theta / np.pi


def montgomery_identity_check(gamma: np.ndarray, alpha: float = 0.65) -> tuple[float, float, float]:
    """Compare␣the␣pair␣sum␣with␣its␣exponentially␣smoothed␣trace␣integral."""
    sample = gamma[:192]
    height = sample[-1]
    frequency = alpha * np.log(height)
    difference = sample[:, None] - sample[None, :]
    direct = np.sum(
        np.exp(1j * frequency * difference) * 4.0 / (4.0 + difference * difference)
    ).real

    grid = np.linspace(-12.0, 12.0, 24001)
    integral_values = np.empty_like(grid)
    for left in range(0, grid.size, 1200):
        right = min(left + 1200, grid.size)
        trace = np.exp(1j * np.outer(frequency + grid[left:right], sample)).sum(axis=1)
        integral_values[left:right] = np.exp(-2.0 * np.abs(grid[left:right])) * np.abs(trace) ** 2
    integrated = float(np.trapezoid(integral_values, grid))
    return float(direct), integrated, abs(float(direct) - integrated)


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, default=root / "data" / "g_hp.bin")
    parser.add_argument(
        "--figure",
        type=Path,
        default=root / "figures" / "core_theorem_compatibility.png",
    )
    args = parser.parse_args()

    gamma = load_float64(args.input)
    count = gamma.size
    ordinal = np.arange(1, count + 1, dtype=np.float64)
    zero_based = ordinal - 1.0
    phase = np.exp(1j * gamma)
    walk = np.cumsum(phase)
    walk_energy_prefix = np.cumsum(np.abs(walk) ** 2)
    phase_prefix = np.cumsum(phase)
    ordinal_phase_prefix = np.cumsum(zero_based * phase)
    clock = rsiemann_clock(gamma)
    clock_phase_prefix = np.cumsum(clock * phase)

    sizes = np.unique(
        np.round(np.logspace(np.log10(300), np.log10(count), 72)).astype(int)
    )
    centroids = phase_prefix[sizes - 1] - ordinal_phase_prefix[sizes - 1] / sizes
    exact_clock = (
```

```
    phase_prefix[sizes - 1]
    + (0.5 * phase_prefix[sizes - 1] - clock_phase_prefix[sizes - 1]) / sizes
)
level_gap = np.abs(centroids - exact_clock)
limit_gap = np.abs(centroids - LIMIT)
energy = walk_energy_prefix[sizes - 1] / sizes
trajectory_compensated = energy / np.log(sizes) ** 2
triangular_remainder = np.abs(
    np.abs(centroids) ** 2 - abs(LIMIT) ** 2 - 1.0 / (6.0 * sizes)
)

inversion_errors: list[float] = []
for frequency in (0.25, 0.5, 1.0, np.log(2.0), 2.0):
    current_phase = np.exp(1j * frequency * gamma)
    current_walk = np.cumsum(current_phase)
    current_A = np.cumsum(current_walk)
    first = np.diff(np.concatenate(([0.0j], current_A)))
    second = np.diff(np.concatenate(([0.0j], first)))
    inversion_errors.append(
        max(
            float(np.max(np.abs(first - current_walk))),
            float(np.max(np.abs(second - current_phase))),
        )
    )

direct_size = 512
weights = 1.0 - np.arange(direct_size, dtype=np.float64) / direct_size
small_phase = phase[:direct_size]
triangular_direct = 0.0
trajectory_direct = 0.0
for j in range(direct_size - 1):
    cosines = np.cos(gamma[j] - gamma[j + 1 : direct_size])
    triangular_direct += 2.0 * weights[j] * np.dot(weights[j + 1 :], cosines)
    trajectory_direct += 2.0 * np.dot(weights[j + 1 :], cosines)
triangular_identity = abs(np.dot(weights, small_phase)) ** 2 - np.dot(weights, weights)
small_walk = np.cumsum(small_phase)
trajectory_energy = float(np.mean(np.abs(small_walk) ** 2))
trajectory_identity = trajectory_energy - (direct_size + 1.0) / 2.0
triangular_closure = abs(triangular_direct - triangular_identity)
trajectory_closure = abs(trajectory_direct - trajectory_identity)

pair_value, integral_value, montgomery_error = montgomery_identity_check(gamma)

final_centroid = centroids[-1]
final_clock = exact_clock[-1]
print("Core␣theorem␣compatibility␣checks")
print(f"input␣=␣data/{args.input.name}")
print(f"input␣sha256␣=␣{sha256(args.input)}")
print(f"ordinates␣=␣{count}")
print(f"ordinate␣range␣=␣[{gamma[0]:.12f},␣{gamma[-1]:.12f}]")
print()
print("Level-one␣exact-clock␣reduction")
print(f"C_N␣=␣{final_centroid.real:+.12f}␣{final_centroid.imag:+.12f}i")
print(f"Q_N^(H)␣=␣{final_clock.real:+.12f}␣{final_clock.imag:+.12f}i")
print(f"|C_N-Q_N^(H)|␣=␣{level_gap[-1]:.12e}")
print(f"midpoint␣clock-defect␣sd␣=␣{np.std(ordinal␣-␣0.5␣-␣clock):.9f}")
print()
print("Conditional␣complex␣centroid␣limit")
print(f"L␣=␣{LIMIT.real:+.15f}␣{LIMIT.imag:+.15f}i")
print(f"|C_N-L|␣=␣{limit_gap[-1]:.12e}")
print(f"Re(C_N-L)␣=␣{(final_centroid-LIMIT).real:+.12e}")
print(f"Im(C_N-L)␣=␣{(final_centroid-LIMIT).imag:+.12e}")
print()
print("Pair␣identities␣at␣N=512␣(direct␣O(N^2)␣sum␣versus␣closed␣form)")
print(f"triangular␣pair␣closure␣error␣=␣{triangular_closure:.3e}")
```

```
print(f"trajectory␣pair␣closure␣error␣=␣{trajectory_closure:.3e}")
print(f"full-N␣triangular␣asymptotic␣remainder␣=␣{triangular_remainder[-1]:.9e}")
print(f"full-N␣trajectory␣energy␣E_N␣=␣{energy[-1]:.9f}")
print(f"full-N␣E_N/log(N)^2␣=␣{trajectory_compensated[-1]:.9f}")
print(f"full-N␣max_(n<=N)|S_n|/log(N)␣=␣{np.max(np.abs(walk))/np.log(count):.9f}")
print()
print("Centroid␣inversion␣finite-difference␣closure")
for frequency, error in zip((0.25, 0.5, 1.0, np.log(2.0), 2.0), inversion_errors):
    print(f"u={frequency:.12f}:␣max␣closure␣error␣=␣{error:.3e}")
print()
print("Montgomery␣identity,␣N=192,␣alpha=0.65")
print(f"direct␣weighted␣pair␣sum␣=␣{pair_value:.12f}")
print(f"smoothed␣trace␣integral␣=␣{integral_value:.12f}")
print(f"absolute␣quadrature␣discrepancy␣=␣{montgomery_error:.3e}")

blue = "#0b6fa4"
orange = "#c8681c"
green = "#4a7c59"
gray = "#8a8a8a"
fig, axes = plt.subplots(2, 3, figsize=(13.0, 7.7), dpi=180)

ax = axes[0, 0]
ax.loglog(sizes, level_gap, color=blue, linewidth=1.2)
ax.set_title("Level-one␣clock␣replacement", loc="left")
ax.set_xlabel(r"$N$")
ax.set_ylabel(r"$|C_N-Q_N^{(H)}|$")

ax = axes[0, 1]
ax.semilogx(sizes, centroids.real, color=blue, linewidth=1.1, label=r"$\Re␣C_N$")
ax.semilogx(sizes, centroids.imag, color=green, linewidth=1.1, label=r"$\Im␣C_N$")
ax.axhline(LIMIT.real, color=orange, linewidth=1.0, linestyle="--")
ax.axhline(LIMIT.imag, color=orange, linewidth=1.0, linestyle="--")
ax.set_title("Both␣centroid␣coordinates", loc="left")
ax.set_xlabel(r"$N$")
ax.legend(frameon=False, fontsize=8)

ax = axes[0, 2]
ax.loglog(sizes, triangular_remainder, color=blue, linewidth=1.1, label="triangular-pair␣remainder")
ax.loglog(sizes, limit_gap, color=orange, linewidth=1.1, linestyle="--", label=r"$|C_N-L|$")
ax.set_title("Pair␣cancellation␣forced␣by␣$C_N$", loc="left")
ax.set_xlabel(r"$N$")
ax.legend(frameon=False, fontsize=7.6)

ax = axes[1, 0]
ax.semilogx(sizes, energy, color=blue, linewidth=1.1, label=r"$\mathcal{E}_N$")
ax.semilogx(sizes, np.log(sizes) ** 2, color=orange, linewidth=1.1, linestyle="--", label=r"$\log^2N$")
ax.set_title("Trajectory-energy␣scale", loc="left")
ax.set_xlabel(r"$N$")
ax.legend(frameon=False, fontsize=8)

ax = axes[1, 1]
frequencies = np.array((0.25, 0.5, 1.0, np.log(2.0), 2.0))
ax.semilogy(frequencies, np.maximum(inversion_errors, 1e-18), "o", color=blue)
ax.set_title("Exact␣centroid␣inversion", loc="left")
ax.set_xlabel(r"frequency␣$u$")
ax.set_ylabel("max␣finite-difference␣error")

ax = axes[1, 2]
ax.bar([0, 1], [pair_value, integral_value], color=[blue, green], width=0.62)
ax.set_xticks([0, 1], ["pair␣sum", "trace␣integral"])
ax.set_title("Montgomery␣identity␣($N=192$)", loc="left")
ax.text(0.5, 0.04, f"$|\\Delta|={montgomery_error:.1e}$", transform=ax.transAxes, ha="center", fontsize=8)

for ax in axes.ravel():
    ax.spines["top"].set_visible(False)
```

```
        ax.spines["right"].set_visible(False)
        ax.spines["left"].set_color(gray)
        ax.spines["bottom"].set_color(gray)
        ax.grid(color="#ececec", linewidth=0.55)
        ax.tick_params(labelsize=8)

    fig.suptitle(
        "Numerical␣compatibility␣checks:␣exact␣closures␣and␣finite-height␣asymptotics",
        x=0.01,
        y=1.01,
        ha="left",
        fontsize=12.4,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure␣=␣figures/{args.figure.name}")


if __name__ == "__main__":
    main()
```

```
Core theorem compatibility checks
input = data/g_hp.bin
input sha256 = 80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2
ordinates = 22491
ordinate range = [14.134725141735, 19999.275621078454]

Level-one exact-clock reduction
C_N = +0.777204389314 +0.768179110409i
Q_N^(H) = +0.777328038812 +0.768246161499i
|C_N-Q_N^(H)| = 1.406593295109e-04
midpoint clock-defect sd = 0.297362294

Conditional complex centroid limit
L = +0.776894260793254 +0.767855668864547i
|C_N-L| = 4.481005826362e-04
Re(C_N-L) = +3.101285209060e-04
Im(C_N-L) = +3.234415444553e-04

Pair identities at N=512 (direct O(N^2) sum versus closed form)
triangular pair closure error = 9.663e-13
trajectory pair closure error = 1.108e-12
full-N triangular asymptotic remainder = 9.713774055e-04
full-N trajectory energy E_N = 3.575354169
full-N E_N/log(N)^2 = 0.035604769
full-N max_(n<=N)|S_n|/log(N) = 0.404122685

Centroid inversion finite-difference closure
u=0.250000000000: max closure error = 1.836e-12
u=0.500000000000: max closure error = 1.792e-12
u=1.000000000000: max closure error = 4.566e-12
u=0.693147180560: max closure error = 3.570e-09
u=2.000000000000: max closure error = 7.388e-12

Montgomery identity, N=192, alpha=0.65
direct weighted pair sum = 194.792615792839
smoothed trace integral = 194.792615832027
absolute quadrature discrepancy = 3.919e-08
figure = figures/core_theorem_compatibility.png
```

Appendix E. Python source: connected form factor

```
#!/usr/bin/env python3
"""Connected Montgomery form factor from the supplied zeta ordinates.

The diagonal contribution is exactly one.  The off-diagonal part is computed
from a weighted histogram of normalized positive pair gaps and then cosine
transformed.  This avoids storing an N by N pair matrix while retaining the
Montgomery weight w(u)=4/(4+u^2).
"""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from data_io import load_float64


TWOPI = 2.0 * np.pi


def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, default=root / "data" / "g_hp.bin")
    parser.add_argument(
        "--figure",
        type=Path,
        default=root / "figures" / "connected_form_factor.png",
    )
    parser.add_argument("--max-lag", type=int, default=500)
    parser.add_argument("--gap-bin", type=float, default=0.01)
    parser.add_argument("--alpha-step", type=float, default=0.01)
    args = parser.parse_args()

    gamma = load_float64(args.input)
    count = gamma.size
    height = float(gamma[-1])
    local_log = float(np.log(height / TWOPI))
    max_gap = args.max_lag + 8.0
    edges = np.arange(0.0, max_gap + args.gap_bin, args.gap_bin)
    weighted_histogram = np.zeros(edges.size - 1, dtype=np.float64)

    for lag in range(1, args.max_lag + 1):
        delta = gamma[lag:] - gamma[:-lag]
        normalized = local_log * delta / TWOPI
        weight = 4.0 / (4.0 + delta * delta)
        weighted_histogram += np.histogram(normalized, bins=edges, weights=weight)[0]

    centers = 0.5 * (edges[:-1] + edges[1:])
```

```
alphas = np.arange(0.02, 4.0 + args.alpha_step * 0.5, args.alpha_step)
off = np.empty_like(alphas)
chunk = 40
coefficient = 2.0 / count
for left in range(0, alphas.size, chunk):
    right = min(left + chunk, alphas.size)
    cosine = np.cos(TWOPI * np.outer(alphas[left:right], centers))
    off[left:right] = coefficient * (cosine @ weighted_histogram)

plateau = alphas >= 1.0
plateau_rms = float(np.sqrt(np.mean(off[plateau] ** 2)))
root_scale = float(local_log / np.sqrt(count))
fitted_constant = plateau_rms / root_scale
finite_target = np.where(
    alphas <= 1.0,
    np.exp(-2.0 * alphas * local_log) * local_log + alphas,
    1.0,
)
ramp_mask = (alphas >= 0.20) & (alphas <= 1.0)
ramp_rmse = float(np.sqrt(np.mean(((1.0 + off[ramp_mask]) - finite_target[ramp_mask]) ** 2)))

scaling_rows: list[tuple[int, float, float, float]] = []
scale_alphas = np.arange(1.0, 4.001, 0.02)
for fraction in (0.125, 0.25, 0.5, 1.0):
    sub_count = int(round(count * fraction))
    sub_gamma = gamma[:sub_count]
    sub_height = float(sub_gamma[-1])
    sub_log = float(np.log(sub_height / TWOPI))
    sub_edges = np.arange(0.0, args.max_lag + 8.0 + args.gap_bin, args.gap_bin)
    sub_hist = np.zeros(sub_edges.size - 1, dtype=np.float64)
    for lag in range(1, min(args.max_lag, sub_count - 1) + 1):
        delta = sub_gamma[lag:] - sub_gamma[:-lag]
        normalized = sub_log * delta / TWOPI
        weight = 4.0 / (4.0 + delta * delta)
        sub_hist += np.histogram(normalized, bins=sub_edges, weights=weight)[0]
    sub_centers = 0.5 * (sub_edges[:-1] + sub_edges[1:])
    sub_off = np.empty_like(scale_alphas)
    for left in range(0, scale_alphas.size, chunk):
        right = min(left + chunk, scale_alphas.size)
        cosine = np.cos(TWOPI * np.outer(scale_alphas[left:right], sub_centers))
        sub_off[left:right] = (2.0 / sub_count) * (cosine @ sub_hist)
    sub_rms = float(np.sqrt(np.mean(sub_off**2)))
    scaling_rows.append((sub_count, sub_log, sub_rms, sub_rms * np.sqrt(sub_count) / sub_log))

print("Connected Montgomery form factor")
print(f"input = data/{args.input.name}")
print(f"input sha256 = {file_sha256(args.input)}")
print(f"ordinates = {count}")
print(f"terminal height = {height:.12f}")
print(f"log(T/(2*pi)) = {local_log:.9f}")
print(f"max ordinal lag = {args.max_lag}")
print(f"normalized-gap bin width = {args.gap_bin:.4f}")
print(f"plateau RMS of Off(alpha), 1<=alpha<=4 = {plateau_rms:.6f}")
print(f"log(T/(2*pi))/sqrt(N) = {root_scale:.6f}")
print(f"plateau RMS coefficient = {fitted_constant:.4f}")
print(f"finite-ramp RMSE, 0.2<=alpha<=1 = {ramp_rmse:.6f}")
print("selected values:")
print("  alpha          Off(alpha)        F(alpha)")
for alpha in (0.10, 0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 2.00, 3.00, 4.00):
    index = int(np.argmin(np.abs(alphas - alpha)))
    print(f"  {alpha:4.2f}          {off[index]:+10.6f}          {1.0 + off[index]:10.6f}")
print("plateau scaling over prefix tables:")
print("  N          log(T/(2*pi))          RMS Off          RMS*sqrt(N)/log")
for sub_count, sub_log, sub_rms, coefficient_row in scaling_rows:
    print(f"  {sub_count:6d}          {sub_log:10.6f}          {sub_rms:9.6f}          {coefficient_row:8.4f}")
```

```
    blue = "#0b6fa4"
    orange = "#c8681c"
    gray = "#8a8a8a"
    fig, axes = plt.subplots(1, 3, figsize=(14.4, 4.7), dpi=180)

    ax = axes[0]
    ax.plot(alphas, 1.0 + off, color=blue, linewidth=1.25, label="computed $F_T$")
    ax.plot(alphas, finite_target, color=orange, linewidth=1.25, linestyle="--", label="ramp / plateau target")

    ax.axvline(1.0, color=orange, linewidth=1.0, linestyle="--")
    ax.text(1.03, 0.80 * np.max(1.0 + off), r"$\alpha=1$", color=orange, fontsize=9)
    ax.set_xlim(0.0, 4.05)
    ax.set_xlabel(r"$\alpha$")
    ax.set_ylabel(r"$F_T(\alpha)$")
    ax.set_title("Ramp and plateau on one scale", loc="left")
    ax.legend(frameon=False, fontsize=7.8)

    ax = axes[1]
    ax.plot(alphas[plateau], off[plateau], color=blue, linewidth=1.25, label=r"$\mathrm{Off}(\alpha)$")
    envelope = fitted_constant * root_scale
    ax.axhline(envelope, color=orange, linewidth=1.0, linestyle="--")
    ax.axhline(-envelope, color=orange, linewidth=1.0, linestyle="--", label=rf"$\pm {fitted_constant:.2f}L_T/\
        sqrt{{N}}$")
    ax.axhline(0.0, color=gray, linewidth=0.8)
    ax.set_xlim(1.0, 4.05)
    bound = max(0.09, 1.15 * float(np.max(np.abs(off[plateau]))))
    ax.set_ylim(-bound, bound)
    ax.set_xlabel(r"$\alpha$")
    ax.set_title("Expanded plateau: root-scale fluctuations", loc="left")
    ax.legend(frameon=False, fontsize=8.5)

    ax = axes[2]
    scale_n = np.array([row[0] for row in scaling_rows], dtype=float)
    scale_coeff = np.array([row[3] for row in scaling_rows])
    ax.semilogx(scale_n, scale_coeff, "o-", color=blue, linewidth=1.15, markersize=4)
    ax.axhline(np.mean(scale_coeff), color=orange, linewidth=1.0, linestyle="--")
    ax.set_xlabel(r"prefix size $N$")
    ax.set_ylabel(r"RMS$(\mathrm{Off})\sqrt{N}/L_T$")
    ax.set_title("Finite-height root-scale check", loc="left")

    for ax in axes:
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.spines["left"].set_color(gray)
        ax.spines["bottom"].set_color(gray)
        ax.grid(color="#ececec", linewidth=0.6)

    fig.suptitle(
        f"Connected form factor from {count:,} zeta ordinates: diagonal = 1 exactly",
        x=0.01,
        y=1.02,
        ha="left",
        fontsize=12.2,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure = figures/{args.figure.name}")


if __name__ == "__main__":
    main()
```

Appendix F. Output: connected form factor

```
Connected Montgomery form factor
input = data/g_hp.bin
input sha256 = 80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2
ordinates = 22491
terminal height = 19999.275621078454
log(T/(2*pi)) = 8.065574267
max ordinal lag = 500
normalized-gap bin width = 0.0100
plateau RMS of Off(alpha), 1<=alpha<=4 = 0.019680
log(T/(2*pi))/sqrt(N) = 0.053781
plateau RMS coefficient = 0.3659
finite-ramp RMSE, 0.2<=alpha<=1 = 0.054321
selected values:
  alpha Off(alpha) F(alpha)
  0.10 +0.538975 1.538975
  0.25 -0.625376 0.374624
  0.50 -0.462208 0.537792
  0.75 -0.172110 0.827890
  1.00 +0.042061 1.042061
  1.25 +0.019714 1.019714
  1.50 +0.008867 1.008867
  2.00 -0.015371 0.984629
  3.00 -0.000053 0.999947
  4.00 -0.019568 0.980432
plateau scaling over prefix tables:
  N log(T/(2*pi)) RMS Off RMS*sqrt(N)/log
    2811 6.277301 0.038853 0.3282
    5623 6.865380 0.032949 0.3599
   11246 7.461733 0.023658 0.3362
   22491 8.065574 0.019844 0.3690
figure = figures/connected_form_factor.png
```

Appendix G. Python source: residual pair law

```
#!/usr/bin/env python3
"""Exact␣RVM-residual␣encoding␣of␣unfolded␣zeta-zero␣pair␣statistics.

The␣script␣uses␣the␣independently␣generated␣ordinate␣table␣supplied␣in␣the
``rvm_portador_residuo``␣package.␣␣It␣verifies␣the␣exact␣identity

␣␣␣␣M(gamma_{k+h})␣-␣M(gamma_k)
␣␣␣␣␣␣=␣h␣+␣eta_{k+h}␣-␣eta_k,

where␣eta_k␣=␣M(gamma_k)␣-␣(k␣-␣1/2),␣estimates␣the␣positive␣pair␣density␣from
the␣residual␣increments,␣and␣compares␣it␣with␣the␣GUE␣sine-kernel␣density.␣␣It
also␣computes␣a␣block-averaged␣unfolded␣form␣factor.␣␣The␣numerical␣comparison
is␣descriptive;␣the␣algebraic␣residual␣identity␣is␣exact␣up␣to␣floating-point
roundoff.
"""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
```

```
from data_io import load_float64


TWOPI = 2.0 * np.pi


def rvm_clock(t: np.ndarray) -> np.ndarray:
    """Leading Riemann--von Mangoldt clock used for exact unfolding."""
    return (t / TWOPI) * np.log(t / (TWOPI * np.e)) + 7.0 / 8.0


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def positive_pair_density(
    unfolded: np.ndarray,
    residual: np.ndarray,
    start: int,
    xmax: float,
    bin_width: float,
    max_lag: int,
) -> tuple[np.ndarray, np.ndarray, float, int]:
    """Estimate the positive two-point density from exact residual increments."""
    edges = np.arange(0.0, xmax + bin_width * 0.5, bin_width)
    counts = np.zeros(edges.size - 1, dtype=np.float64)
    population = unfolded.size - start
    reconstruction_error = 0.0

    for lag in range(1, max_lag + 1):
        direct = unfolded[start + lag :] - unfolded[start:-lag]
        reconstructed = lag + residual[start + lag :] - residual[start:-lag]
        reconstruction_error = max(
            reconstruction_error,
            float(np.max(np.abs(direct - reconstructed))),
        )
        counts += np.histogram(reconstructed, bins=edges)[0]

    centers = 0.5 * (edges[:-1] + edges[1:])
    density = counts / (population * bin_width)
    return centers, density, reconstruction_error, population


def windowed_connected_form_factor(
    centers: np.ndarray,
    density: np.ndarray,
    reference_density: np.ndarray,
    bin_width: float,
    xmax: float,
    alphas: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    """Cosine transform of the connected pair density with a Hann gap window.

    A compact gap window is essential here.  The raw finite trace contains
    coherent arithmetic/clock spikes and is not a pointwise estimator of the
    GUE form factor.  Applying the same window to the data and the sine-kernel
    reference gives a like-for-like connected statistic.
    """
    window = 0.5 * (1.0 + np.cos(np.pi * centers / xmax))
    cosine = np.cos(TWOPI * np.outer(alphas, centers))
```

```
    measured = 1.0 + 2.0 * bin_width * (cosine @ (window * (density - 1.0)))
    reference = 1.0 + 2.0 * bin_width * (
        cosine @ (window * (reference_density - 1.0))
    )
    return measured, reference


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    default_input = root / "data" / "g_hp.bin"
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, default=default_input)
    parser.add_argument(
        "--figure",
        type=Path,
        default=root / "figures" / "unfolded_residual_pair_law.png",
    )
    parser.add_argument("--burn", type=int, default=1200)
    parser.add_argument("--xmax", type=float, default=8.0)
    parser.add_argument("--bin-width", type=float, default=0.08)
    parser.add_argument("--max-lag", type=int, default=16)
    args = parser.parse_args()

    gammas = load_float64(args.input)
    ordinal = np.arange(1, gammas.size + 1, dtype=np.float64)
    unfolded = rvm_clock(gammas)
    residual = unfolded - (ordinal - 0.5)

    centers, density, identity_error, population = positive_pair_density(
        unfolded,
        residual,
        args.burn,
        args.xmax,
        args.bin_width,
        args.max_lag,
    )
    sine_density = 1.0 - np.sinc(centers) ** 2
    comparison = (centers >= 0.2) & (centers <= 6.0)
    pair_rmse = float(np.sqrt(np.mean((density[comparison] - sine_density[comparison]) ** 2)))

    alpha_grid = np.unique(
        np.concatenate(
            [
                np.linspace(0.05, 2.0, 391),
                np.array([0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 2.00]),
            ]
        )
    )
    form_factor, windowed_gue = windowed_connected_form_factor(
        centers,
        density,
        sine_density,
        args.bin_width,
        args.xmax,
        alpha_grid,
    )
    form_mask = (alpha_grid >= 0.10) & (alpha_grid <= 1.80)
    form_rmse = float(
        np.sqrt(np.mean((form_factor[form_mask] - windowed_gue[form_mask]) ** 2))
    )

    tail_residual = residual[args.burn :]
    tail_size = tail_residual.size
    integer_rows: list[tuple[int, float, float]] = []
    for mode in (1, 2, 3):
```

```
        residual_sum = abs(np.sum(np.exp(1j * TWOPI * mode * tail_residual)))
        integer_rows.append((mode, residual_sum, residual_sum / np.sqrt(tail_size)))

    try:
        displayed_input = args.input.resolve().relative_to(root.resolve())
    except ValueError:
        displayed_input = args.input
    try:
        displayed_figure = args.figure.resolve().relative_to(root.resolve())
    except ValueError:
        displayed_figure = args.figure

    print("Exact␣unfolded␣RVM-residual␣pair␣analysis")
    print(f"input␣=␣{displayed_input}")
    print(f"input␣sha256␣=␣{sha256(args.input)}")
    print(f"ordinates␣=␣{gammas.size}")
    print(f"ordinate␣range␣=␣[{gammas[0]:.12f},␣{gammas[-1]:.12f}]")
    print(f"analysis␣tail␣starts␣at␣ordinal␣=␣{args.burn␣+␣1}")
    print(
        "eta␣statistics:␣"
        f"mean={np.mean(tail_residual):+.6e},␣"
        f"sd={np.std(tail_residual):.6f},␣"
        f"min={np.min(tail_residual):+.6f},␣"
        f"max={np.max(tail_residual):+.6f}"
    )
    print(
        "max␣reconstruction␣error␣in␣"
        "M(gamma[k+h])-M(gamma[k])␣=␣h+eta[k+h]-eta[k]:␣"
        f"{identity_error:.3e}"
    )
    print(
        f"positive␣pair␣density:␣population={population},␣max_lag={args.max_lag},␣"
        f"bin_width={args.bin_width:.3f},␣RMSE␣on␣0.2<=s<=6␣=␣{pair_rmse:.4f}"
    )
    print(
        "Hann-windowed␣connected␣form␣factor:␣"
        f"gap␣cutoff={args.xmax:.1f},␣RMSE␣on␣0.1<=alpha<=1.8␣=␣{form_rmse:.4f}"
    )
    print("integer␣residual␣modes␣(the␣unfolded␣ordinal␣carrier␣is␣constant):")
    print("␣␣m␣␣␣␣␣␣␣|sum␣exp(2*pi*i*m*eta_k)|␣␣␣␣␣␣/sqrt(H)")
    for mode, magnitude, normalized in integer_rows:
        print(f"␣␣{mode:d}␣␣␣␣␣␣␣{magnitude:18.6f}␣␣␣␣␣␣{normalized:10.6f}")
    print("selected␣windowed␣connected␣form-factor␣values:")
    print("␣␣alpha␣␣␣␣␣␣␣␣measured␣␣␣␣␣␣windowed␣GUE")
    for alpha in (0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 2.00):
        index = int(np.argmin(np.abs(alpha_grid - alpha)))
        print(
            f"␣␣{alpha:4.2f}␣␣␣␣␣␣␣␣{form_factor[index]:8.4f}␣␣␣␣␣␣␣␣"
            f"{windowed_gue[index]:8.4f}"
        )

    ink = "#202020"
    blue = "#0b6fa4"
    orange = "#c8681c"
    green = "#4a7c59"
    gray = "#8a8a8a"
    fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.7), dpi=180)

    ax = axes[0]
    ax.plot(centers, density, color=blue, linewidth=1.25, label="residual-increment␣estimate")
    ax.plot(
        centers,
        sine_density,
        color=orange,
        linewidth=1.8,
```

```
        linestyle="--",
        label=r"$1-[\sin(\pi␣s)/(\pi␣s)]^2$",
    )
    ax.set_xlim(0.0, args.xmax)
    ax.set_ylim(0.0, 1.35)
    ax.set_xlabel(r"unfolded␣positive␣separation␣$s$")
    ax.set_ylabel(r"pair␣density␣$R_2(s)$")
    ax.set_title("The␣sine␣law␣is␣encoded␣by␣RVM-residual␣increments", loc="left")
    ax.legend(frameon=False, fontsize=8.5)

    ax = axes[1]
    ax.plot(
        alpha_grid,
        form_factor,
        color=blue,
        linewidth=1.2,
        label="residual-increment␣transform",
    )
    ax.plot(
        alpha_grid,
        windowed_gue,
        color=green,
        linewidth=1.8,
        linestyle="--",
        label="same␣transform␣of␣the␣sine␣law",
    )
    ax.axvline(1.0, color=gray, linewidth=0.8, linestyle=":")
    ax.set_xlim(0.05, 2.0)
    ax.set_ylim(0.0, max(1.4, float(np.quantile(form_factor, 0.99)) * 1.1))
    ax.set_xlabel(r"unfolded␣frequency␣$\alpha$")
    ax.set_ylabel("windowed␣connected␣form␣factor")
    ax.set_title("Fourier-dual␣check␣after␣gap␣smoothing", loc="left")
    ax.legend(frameon=False, fontsize=8.5)

    for ax in axes:
        ax.tick_params(labelsize=8.5)
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.spines["left"].set_color(gray)
        ax.spines["bottom"].set_color(gray)
        ax.grid(color="#ececec", linewidth=0.6, zorder=0)

    fig.suptitle(
        "Exact␣residual␣unfolding␣of␣22,491␣computed␣zeta␣ordinates",
        x=0.01,
        y=1.02,
        ha="left",
        color=ink,
        fontsize=12.0,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure␣=␣{displayed_figure}")


if __name__ == "__main__":
    main()
```

Appendix H. Output: residual pair law

```
Exact unfolded RVM-residual pair analysis
input = data/g_hp.bin
```

```
input sha256 = 80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2
ordinates = 22491
ordinate range = [14.134725141735, 19999.275621078454]
analysis tail starts at ordinal = 1201
eta statistics: mean=-1.298000e-06, sd=0.299245, min=-0.948472, max=+0.957006
max reconstruction error in M(gamma[k+h])-M(gamma[k]) = h+eta[k+h]-eta[k]: 0.000e+00
positive pair density: population=21291, max_lag=16, bin_width=0.080, RMSE on 0.2<=s<=6 = 0.0362
Hann-windowed connected form factor: gap cutoff=8.0, RMSE on 0.1<=alpha<=1.8 = 0.0410
integer residual modes (the unfolded ordinal carrier is constant):
  m |sum exp(2*pi*i*m*eta_k)| /sqrt(H)
  1 2907.775976 19.927963
  2 79.116220 0.542210
  3 63.535015 0.435427
selected windowed connected form-factor values:
  alpha measured windowed GUE
  0.25 0.2140 0.2500
  0.50 0.4633 0.5000
  0.75 0.7302 0.7500
  1.00 0.9830 0.9816
  1.25 1.0809 1.0000
  1.50 1.0167 1.0000
  2.00 1.0042 1.0000
figure = figures/unfolded_residual_pair_law.png
```

Appendix I. Python source: centroid-rate validation

```
#!/usr/bin/env python3
"""Independent␣finite-range␣validation␣of␣the␣log(N)/N␣centroid␣rate."""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from data_io import load_float64


LIMIT = 0.77689426079325358 + 0.767855668864547j
FULL_COUNT = 2_001_052
FULL_OBSERVED = 8.7162e-6


def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, default=root / "data" / "g_hp.bin")
    parser.add_argument(
        "--figure",
```

```
    type=Path,
    default=root / "figures" / "centroid_rate_validation.png",
)
args = parser.parse_args()

gamma = load_float64(args.input)
phase = np.exp(1j * gamma)
prefix = np.concatenate(([0.0j], np.cumsum(phase)))
weighted_prefix = np.concatenate(
    ([0.0j], np.cumsum(np.arange(gamma.size, dtype=np.float64) * phase))
)

def centroid(size: int) -> complex:
    return prefix[size] - weighted_prefix[size] / size

sizes = np.unique(
    np.round(np.logspace(np.log10(300), np.log10(gamma.size), 60)).astype(int)
)
error = np.array([abs(centroid(int(size)) - LIMIT) for size in sizes])
compensated = error * sizes / np.log(sizes)
coefficient = float(np.mean(compensated))
coefficient_sd = float(np.std(compensated))
prediction = coefficient * np.log(FULL_COUNT) / FULL_COUNT

print("Independent␣centroid-rate␣validation")
print(f"input␣=␣data/{args.input.name}")
print(f"input␣sha256␣=␣{file_sha256(args.input)}")
print(f"ordinates␣=␣{gamma.size}")
print(f"conditional␣limit␣=␣{LIMIT.real:.17f}␣{LIMIT.imag:+.15f}i")
print(f"mean␣of␣|C_N-L|*N/log(N)␣=␣{coefficient:.6f}")
print(f"standard␣deviation␣=␣{coefficient_sd:.6f}")
print(f"minimum␣=␣{np.min(compensated):.6f}")
print(f"maximum␣=␣{np.max(compensated):.6f}")
print(f"prediction␣at␣N={FULL_COUNT}␣=␣{prediction:.8e}")
print(f"full-table␣observed␣discrepancy␣=␣{FULL_OBSERVED:.8e}")
print(f"observed/predicted␣=␣{FULL_OBSERVED␣/␣prediction:.6f}")

blue = "#0b6fa4"
orange = "#c8681c"
green = "#4a7c59"
gray = "#8a8a8a"
fig, axes = plt.subplots(1, 2, figsize=(11.6, 4.5), dpi=180)

ax = axes[0]
ax.loglog(sizes, error, color=blue, linewidth=1.25, label=r"$|C_N-L|$")
ax.loglog(
    sizes,
    coefficient * np.log(sizes) / sizes,
    color=orange,
    linewidth=1.6,
    linestyle="--",
    label=rf"${coefficient:.3f}\log␣N/N$",
)
root_coefficient = float(np.mean(error * np.sqrt(sizes)))
ax.loglog(
    sizes,
    root_coefficient / np.sqrt(sizes),
    color=green,
    linewidth=1.2,
    linestyle=":",
    label=r"best␣$N^{-1/2}$␣comparison",
)
ax.set_xlabel(r"$N$")
ax.set_ylabel(r"$|C_N-L|$")
ax.set_title("Independent␣low-height␣ordinate␣table", loc="left")
```

```
    ax.legend(frameon=False, fontsize=8.3)

    ax = axes[1]
    ax.semilogx(sizes, compensated, color=orange, linewidth=1.1)
    ax.axhline(coefficient, color=gray, linewidth=0.8, linestyle="--")
    ax.scatter(
        [FULL_COUNT],
        [FULL_OBSERVED * FULL_COUNT / np.log(FULL_COUNT)],
        color=blue,
        s=35,
        zorder=5,
        label=r"independent␣full-table␣check,␣$N=2{,}001{,}052$",
    )
    ax.set_xlabel(r"$N$")
    ax.set_ylabel(r"$|C_N-L|N/\log␣N$")
    ax.set_title("Compensated␣rate␣and␣90-fold␣extrapolation", loc="left")
    ax.legend(frameon=False, fontsize=8.2)

    for ax in axes:
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.spines["left"].set_color(gray)
        ax.spines["bottom"].set_color(gray)
        ax.grid(color="#ececec", linewidth=0.6)

    fig.suptitle(
        r"Observed␣exponential-centroid␣rate:␣$|C_N-L|\asymp␣\log␣N/N$",
        x=0.01,
        y=1.02,
        ha="left",
        fontsize=12.2,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure␣=␣figures/{args.figure.name}")


if __name__ == "__main__":
    main()
```

Appendix J. Output: centroid-rate validation

```
Independent centroid-rate validation
input = data/g_hp.bin
input sha256 = 80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2
ordinates = 22491
conditional limit = 0.77689426079325363 +0.767855668864547i
mean of |C_N-L|*N/log(N) = 1.144738
standard deviation = 0.258893
minimum = 0.665957
maximum = 1.764019
prediction at N=2001052 = 8.30024047e-06
full-table observed discrepancy = 8.71620000e-06
observed/predicted = 1.050114
figure = figures/centroid_rate_validation.png
```

Appendix K. Python source and output: carrier saddle spectrum

```
#!/usr/bin/env python3
"""Numerically␣verify␣the␣localized␣archimedean␣stationary-phase␣law."""
```

```
from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from data_io import load_float64


TWOPI = 2.0 * np.pi


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def carrier_amplitude(tau: np.ndarray, alpha: float, logarithm: float) -> float:
    phase = np.longdouble(alpha * logarithm) * tau.astype(np.longdouble)
    two_pi = np.longdouble(2.0) * np.longdouble(np.pi)
    phase -= two_pi * np.floor(phase / two_pi)
    return float(abs(np.sum(np.exp(1j * phase.astype(np.float64)))))


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--input", type=Path, default=root / "data" / "rvm_tau_2001052.bin"
    )
    parser.add_argument(
        "--figure",
        type=Path,
        default=root / "figures" / "carrier_saddle_validation.png",
    )
    args = parser.parse_args()
    tau = load_float64(args.input)
    count = tau.size
    height = float(tau[-1])
    logarithm = float(np.log(height / TWOPI))

    # The full binary contains 2,001,052 phases. A 13-point grid in each
    # nonintegral band keeps the validation reproducible without replacing the
    # actual sum by a subsample.
    intervals = [
        np.linspace(0.22, 0.94, 13),
        np.linspace(1.06, 1.94, 13),
        np.linspace(2.06, 2.94, 13),
    ]
    alphas = np.concatenate(intervals)
    measured = np.array([carrier_amplitude(tau, float(a), logarithm) for a in alphas])
    mode = np.ceil(alphas).astype(int)
    time = alphas * logarithm
    predicted = time * np.exp(time / (2.0 * mode)) / mode ** 1.5
    ratio = measured / predicted
```

```
selected = np.array([0.30, 0.50, 0.70, 0.90, 1.20, 1.40, 1.80, 2.20, 2.60, 2.90])
selected_measured = np.array([
    measured[int(np.argmin(np.abs(alphas - a)))]
    if np.min(np.abs(alphas - a)) < 1e-12
    else carrier_amplitude(tau, float(a), logarithm)
    for a in selected
])
selected_mode = np.ceil(selected).astype(int)
selected_time = selected * logarithm
selected_predicted = (
    selected_time * np.exp(selected_time / (2.0 * selected_mode)) / selected_mode ** 1.5
)

print("Archimedean␣carrier␣saddle␣validation")
print(f"input␣=␣data/{args.input.name}")
print(f"input␣sha256␣=␣{sha256(args.input)}")
print(f"count␣=␣{count}")
print(f"tau_N␣=␣{height:.12f}")
print(f"log(tau_N/(2*pi))␣=␣{logarithm:.9f}")
print("stationary␣phase:␣tau_n^*=2*pi*exp(t/n)")
print("leading␣magnitude:␣t*exp(t/(2*n))/n^(3/2),␣n=ceil(alpha)")
print("␣␣alpha␣␣␣␣mode␣␣␣␣␣␣␣measured␣␣␣␣␣␣␣predicted␣␣␣␣␣␣␣ratio")
for alpha, nmode, actual, estimate in zip(
    selected, selected_mode, selected_measured, selected_predicted
):
    print(f"␣␣{alpha:5.2f}␣␣␣␣␣{nmode:2d}␣␣␣␣␣{actual:11.3f}␣␣␣␣␣{estimate:11.3f}␣␣␣␣␣{actual/estimate:8.4f}")
interior = np.minimum(alphas - np.floor(alphas), np.ceil(alphas) - alphas) >= 0.08
print(f"median␣measured/predicted␣away␣from␣integer␣endpoints␣=␣{np.median(ratio[interior]):.6f}")
print(f"RMS␣relative␣error␣away␣from␣integer␣endpoints␣=␣{np.sqrt(np.mean((ratio[interior]-1.0)**2)):.6f}")


blue = "#0b6fa4"
orange = "#c8681c"
gray = "#8a8a8a"
fig, axes = plt.subplots(1, 2, figsize=(11.8, 4.55), dpi=180)
ax = axes[0]
ax.semilogy(alphas, measured, color=blue, linewidth=1.05, label="measured␣carrier␣sum")
ax.semilogy(alphas, predicted, color=orange, linewidth=1.25, linestyle="--", label="localized␣saddle␣
    magnitude")
for endpoint in (1.0, 2.0, 3.0):
    ax.axvline(endpoint, color=gray, linewidth=0.7, linestyle=":")
ax.set_xlabel(r"$\alpha$,␣with␣$t=\alpha\log(T/2\pi)$")
ax.set_ylabel(r"magnitude")
ax.set_title("Closed␣carrier␣spectrum␣between␣endpoints", loc="left")
ax.legend(frameon=False, fontsize=8.2)

ax = axes[1]
ax.plot(alphas, ratio, color=blue, linewidth=1.05)
ax.axhline(1.0, color=orange, linewidth=1.0, linestyle="--")
for endpoint in (1.0, 2.0, 3.0):
    ax.axvline(endpoint, color=gray, linewidth=0.7, linestyle=":")
ax.set_ylim(0.55, 1.45)
ax.set_xlabel(r"$\alpha$")
ax.set_ylabel("measured␣/␣leading␣saddle")
ax.set_title("Finite-$N$␣compatibility␣of␣the␣amplitude", loc="left")

for ax in axes:
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["left"].set_color(gray)
    ax.spines["bottom"].set_color(gray)
    ax.grid(color="#ececec", linewidth=0.55)
fig.suptitle(
    f"Lambert--RVM␣archimedean␣saddles,␣N={count:,}",
    x=0.01,
```

```
        y=1.02,
        ha="left",
        fontsize=12.2,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure␣=␣figures/{args.figure.name}")


if __name__ == "__main__":
    main()
```

```
Archimedean carrier saddle validation
input = data/rvm_tau_2001052.bin
input sha256 = ae9d35415ee924504638d96e9778a2f63f20e9c1d2ae14ce3ecf35f775bce57a
count = 2001052
tau_N = 1132490.722587621538
log(tau_N/(2*pi)) = 12.102052878
stationary phase: tau_n^*=2*pi*exp(t/n)
leading magnitude: t*exp(t/(2*n))/n^(3/2), n=ceil(alpha)
  alpha mode measured predicted ratio
   0.30 1 23.530 22.303 1.0550
   0.50 1 123.073 124.679 0.9871
   0.70 1 554.283 585.472 0.9467
   0.90 1 2539.974 2524.854 1.0060
   1.20 2 194.367 193.754 1.0032
   1.40 2 429.149 413.991 1.0366
   1.80 2 1687.315 1785.341 0.9451
   2.20 3 387.241 433.258 0.8938
   2.60 3 948.686 1147.328 0.8269
   2.90 3 2263.552 2343.716 0.9658
median measured/predicted away from integer endpoints = 0.982468
RMS relative error away from integer endpoints = 0.113034
figure = figures/carrier_saddle_validation.png
```

Appendix L. Python source and output: Lambert counterexample

```
#!/usr/bin/env python3
"""Finite␣verification␣of␣the␣Lambert-centroid/Dirac-comb␣counterexample."""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from data_io import load_float64


TWOPI = 2.0 * np.pi


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
```

```
            digest.update(chunk)
    return digest.hexdigest()


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--input", type=Path, default=root / "data" / "rvm_tau_2001052.bin"
    )
    parser.add_argument(
        "--figure",
        type=Path,
        default=root / "figures" / "rvm_counterexample_validation.png",
    )
    args = parser.parse_args()
    tau = load_float64(args.input)
    count = tau.size
    phase = np.exp(1j * tau)
    prefix = np.cumsum(phase)
    weighted = np.cumsum(np.arange(count, dtype=np.float64) * phase)

    def centroid(size: int) -> complex:
        return prefix[size - 1] - weighted[size - 1] / size

    base_sizes = np.unique(
        np.round(np.logspace(np.log10(1000), np.log10(count // 2), 64)).astype(int)
    )
    increments = np.array([abs(centroid(2 * n) - centroid(n)) for n in base_sizes])
    compensated = increments * base_sizes / np.log(base_sizes) ** 2

    ordinal = np.arange(1, count + 1, dtype=np.float64)
    clock_residual = (
        (tau / TWOPI) * np.log(tau / (TWOPI * np.e))
        + 7.0 / 8.0
        - (ordinal - 0.5)
    )

    print("Lambert␣centroid␣and␣lattice-pair␣counterexample")
    print(f"input␣=␣data/{args.input.name}")
    print(f"input␣sha256␣=␣{sha256(args.input)}")
    print(f"levels␣=␣{count}")
    print(f"max␣float64␣|M(tau_k)-(k-1/2)|␣=␣{np.max(np.abs(clock_residual)):.3e}")
    print(f"C_N^M␣=␣{centroid(count).real:+.12f}␣{centroid(count).imag:+.12f}i")
    print("successive-doubling␣convergence␣check")
    print(f"mean␣|C_(2N)-C_N|*N/log(N)^2␣=␣{np.mean(compensated):.9f}")
    print(f"sd␣=␣{np.std(compensated):.9f}")
    print(f"min␣=␣{np.min(compensated):.9f}")
    print(f"max␣=␣{np.max(compensated):.9f}")
    print("unfolded␣pair␣locations␣=␣positive␣integers␣exactly␣in␣the␣defining␣clock")
    print("comparison␣process␣=␣continuous␣1-sinc(s)^2␣density")

    blue = "#0b6fa4"
    orange = "#c8681c"
    green = "#4a7c59"
    gray = "#8a8a8a"
    fig, axes = plt.subplots(1, 2, figsize=(11.8, 4.55), dpi=180)

    ax = axes[0]
    ax.loglog(base_sizes, increments, color=blue, linewidth=1.15, label=r"$|C_{2N}^M-C_N^M|$")
    coefficient = float(np.median(compensated))
    ax.loglog(
        base_sizes,
        coefficient * np.log(base_sizes) ** 2 / base_sizes,
        color=orange,
```

```
        linewidth=1.3,
        linestyle="--",
        label=rf"${coefficient:.3f}\log^2N/N$",
    )
    ax.set_xlabel(r"$N$")
    ax.set_ylabel("successive-doubling␣difference")
    ax.set_title("Lambert␣centroid:␣convergence-scale␣check", loc="left")
    ax.legend(frameon=False, fontsize=8.2)

    ax = axes[1]
    block = 200
    integer = np.arange(1, 9, dtype=np.float64)
    lattice_mass = 1.0 - integer / block
    ax.bar(integer, lattice_mass, width=0.16, color=blue, alpha=0.78, label="RVM␣pair␣atoms")
    continuum = np.linspace(0.02, 8.5, 1200)
    ax.plot(
        continuum,
        1.0 - np.sinc(continuum) ** 2,
        color=green,
        linewidth=1.65,
        label="GUE␣sine␣density",
    )
    ax.set_xlim(0.0, 8.5)
    ax.set_ylim(0.0, 1.12)
    ax.set_xlabel("positive␣unfolded␣separation")
    ax.set_ylabel("atom␣mass␣/␣comparison␣density")
    ax.set_title("Exact␣obstruction:␣Dirac␣comb,␣not␣sine␣law", loc="left")
    ax.legend(frameon=False, fontsize=8.2)

    for ax in axes:
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.spines["left"].set_color(gray)
        ax.spines["bottom"].set_color(gray)
        ax.grid(color="#ececec", linewidth=0.55)
    fig.suptitle(
        "The␣Lambert␣clock␣separates␣centroid␣convergence␣from␣pair␣correlation",
        x=0.01,
        y=1.02,
        ha="left",
        fontsize=12.2,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure␣=␣figures/{args.figure.name}")


if __name__ == "__main__":
    main()
```

```
Lambert centroid and lattice-pair counterexample
input = data/rvm_tau_2001052.bin
input sha256 = ae9d35415ee924504638d96e9778a2f63f20e9c1d2ae14ce3ecf35f775bce57a
levels = 2001052
max float64 |M(tau_k)-(k-1/2)| = 4.657e-10
C_N^M = +0.041570229566 +1.757541724624i
successive-doubling convergence check
mean |C_(2N)-C_N|*N/log(N)^2 = 0.018281615
sd = 0.008659363
min = 0.000958714
max = 0.036188306
unfolded pair locations = positive integers exactly in the defining clock
comparison process = continuous 1-sinc(s)^2 density
```

```
figure = figures/rvm_counterexample_validation.png
```

Appendix M. Python source and output: truncated prime model

```
#!/usr/bin/env python3
"""Exact finite-sum check of the twisted quadratic prime-polynomial bound."""

from __future__ import annotations

import argparse
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np


def prime_power_terms(limit: int) -> tuple[np.ndarray, np.ndarray]:
    """Return log(n) and Lambda(n)/(sqrt(n) log(n)) for n<=limit."""
    if limit < 2:
        return np.empty(0), np.empty(0)
    sieve = np.ones(limit + 1, dtype=bool)
    sieve[:2] = False
    for prime in range(2, int(limit**0.5) + 1):
        if sieve[prime]:
            sieve[prime * prime :: prime] = False
    primes = np.flatnonzero(sieve)
    entries: list[tuple[int, float]] = []
    for prime in primes:
        power = int(prime)
        exponent = 1
        while power <= limit:
            entries.append((power, 1.0 / (exponent * np.sqrt(power))))
            if power > limit // int(prime):
                break
            power *= int(prime)
            exponent += 1
    entries.sort()
    values = np.array([entry[0] for entry in entries], dtype=np.float64)
    coefficients = np.array([entry[1] for entry in entries], dtype=np.float64)
    return np.log(values), coefficients


def exponential_integral(frequency: np.ndarray, height: float) -> np.ndarray:
    result = np.empty(frequency.shape, dtype=np.complex128)
    small = np.abs(frequency) < 1e-12
    result[small] = height
    omega = frequency[~small]
    result[~small] = np.expm1(1j * omega * height) / (1j * omega)
    return result


def exact_twisted_square(height: float, alpha: float, theta: float) -> tuple[complex, int, int]:
    cutoff = max(2, int(np.floor(height**theta)))
    logarithms, coefficients = prime_power_terms(cutoff)
    a = alpha * np.log(height)
    plus = logarithms[:, None] + logarithms[None, :]
    minus = logarithms[:, None] - logarithms[None, :]
    kernel = 0.25 * (
        exponential_integral(a + minus, height)
        + exponential_integral(a - minus, height)
```

```
        - exponential_integral(a + plus, height)
        - exponential_integral(a - plus, height)
    )
    integral = np.sum((coefficients[:, None] * coefficients[None, :]) * kernel) / np.pi**2
    return complex(integral), cutoff, coefficients.size


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--figure", type=Path, default=root / "figures" / "prime_model_validation.png"
    )
    args = parser.parse_args()

    cases = [
        (0.50, 0.375, 0.10),
        (1.00, 0.750, 0.10),
        (1.50, 0.875, 0.05),
    ]
    heights = np.array([300.0, 800.0, 2000.0, 5000.0, 12000.0])
    records: dict[tuple[float, float], tuple[np.ndarray, np.ndarray]] = {}

    print("Twisted␣prime-polynomial␣finite-sum␣validation")
    print("P_Y(t)=-(1/pi)␣sum_{n<=Y}␣Lambda(n)/(sqrt(n)log(n))*sin(t␣log␣n)")
    print("integrals␣are␣evaluated␣algebraically␣from␣exponential␣antiderivatives")
    for alpha, theta, epsilon in cases:
        if not (alpha / 2.0 < theta < min(alpha, 1.0)):
            raise RuntimeError("invalid␣theta␣range")
        actual: list[float] = []
        normalized_bound: list[float] = []
        print()
        print(f"alpha={alpha:.2f},␣theta={theta:.3f},␣epsilon={epsilon:.3f}")
        print(f"theta-alpha/2␣=␣{theta-alpha/2:.3f};␣min(alpha,1)-theta␣=␣{min(alpha,1)-theta:.3f}")
        print("␣␣␣␣␣␣T␣␣␣␣␣␣␣Y␣␣␣␣␣prime␣powers␣␣␣␣␣␣␣|I(T)|/T␣␣␣␣␣␣␣bound-shape/T␣␣␣␣␣␣ratio")
        for height in heights:
            integral, cutoff, term_count = exact_twisted_square(height, alpha, theta)
            observed = abs(integral) / height
            shape = (
                height ** (1.0 - alpha / 2.0 + epsilon)
                + height ** (alpha / 2.0 + epsilon)
                + height ** (theta + epsilon)
            ) / height
            actual.append(observed)
            normalized_bound.append(shape)
            print(
                f"{height:8.0f}␣␣{cutoff:6d}␣␣␣␣␣{term_count:6d}␣␣␣␣␣␣␣"
                f"{observed:12.5e}␣␣␣␣␣{shape:12.5e}␣␣␣␣{observed/shape:8.4f}"
            )
        records[(alpha, theta)] = (np.array(actual), np.array(normalized_bound))

    blue = "#0b6fa4"
    orange = "#c8681c"
    green = "#4a7c59"
    gray = "#8a8a8a"
    colors = [blue, orange, green]
    fig, axes = plt.subplots(1, 2, figsize=(11.8, 4.55), dpi=180)

    ax = axes[0]
    for (alpha, theta, _), color in zip(cases, colors):
        actual, _ = records[(alpha, theta)]
        ax.loglog(heights, actual, "o-", color=color, linewidth=1.1, markersize=3.8, label=rf"$\alpha={alpha:.1
            f},\␣\theta={theta:.3f}$")
    ax.set_xlabel(r"$T$")
    ax.set_ylabel(r"$T^{-1}|\int_0^T␣e^{i\alpha\log␣T\,t}P_Y(t)^2\,dt|$")
```

```
    ax.set_title("Exact␣finite␣prime-polynomial␣integrals", loc="left")
    ax.legend(frameon=False, fontsize=8)

    ax = axes[1]
    for (alpha, theta, _), color in zip(cases, colors):
        actual, shape = records[(alpha, theta)]
        ax.semilogx(heights, actual / shape, "o-", color=color, linewidth=1.1, markersize=3.8, label=rf"$\alpha
            ={alpha:.1f}$")
    ax.set_xlabel(r"$T$")
    ax.set_ylabel("observed␣/␣stated␣bound␣shape")
    ax.set_title("Compatibility␣with␣all␣three␣exponent␣terms", loc="left")
    ax.legend(frameon=False, fontsize=8)

    for ax in axes:
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.spines["left"].set_color(gray)
        ax.spines["bottom"].set_color(gray)
        ax.grid(color="#ececec", linewidth=0.55)
    fig.suptitle(
        "Twisted␣quadratic␣cancellation␣in␣the␣truncated␣Selberg␣prime␣model",
        x=0.01,
        y=1.02,
        ha="left",
        fontsize=12.2,
    )
    fig.tight_layout()
    args.figure.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(args.figure, bbox_inches="tight", facecolor="white")
    print(f"figure␣=␣figures/{args.figure.name}")


if __name__ == "__main__":
    main()
```

```
Twisted prime-polynomial finite-sum validation
P_Y(t)=-(1/pi) sum_{n<=Y} Lambda(n)/(sqrt(n)log(n))*sin(t log n)
integrals are evaluated algebraically from exponential antiderivatives

alpha=0.50, theta=0.375, epsilon=0.100
theta-alpha/2 = 0.125; min(alpha,1)-theta = 0.125
     T Y prime powers |I(T)|/T bound-shape/T ratio
   300 8 6 2.36036e-04 4.99644e-01 0.0005
   800 12 8 7.77263e-04 4.09776e-01 0.0019
  2000 17 11 1.13466e-04 3.45417e-01 0.0003
  5000 24 13 6.30831e-05 2.94083e-01 0.0002
 12000 33 18 1.96800e-05 2.53862e-01 0.0001

alpha=1.00, theta=0.750, epsilon=0.100
theta-alpha/2 = 0.250; min(alpha,1)-theta = 0.250
     T Y prime powers |I(T)|/T bound-shape/T ratio
   300 72 29 3.56840e-03 6.29301e-01 0.0057
   800 150 48 2.79568e-03 5.04863e-01 0.0055
  2000 299 79 4.59995e-04 4.15411e-01 0.0011
  5000 594 129 2.59604e-04 3.45000e-01 0.0008
 12000 1146 215 6.00727e-04 2.91117e-01 0.0021

alpha=1.50, theta=0.875, epsilon=0.050
theta-alpha/2 = 0.125; min(alpha,1)-theta = 0.125
     T Y prime powers |I(T)|/T bound-shape/T ratio
   300 147 47 2.42370e-03 9.89980e-01 0.0024
   800 346 86 1.46832e-03 8.77654e-01 0.0017
  2000 773 160 1.14023e-03 7.89049e-01 0.0014
  5000 1724 298 6.48359e-04 7.12562e-01 0.0009
```

```
   12000 3709 556 4.52507e-04 6.48590e-01 0.0007
figure = figures/prime_model_validation.png
```

Appendix N. Python source and output: same-scale 3D clouds

```
#!/usr/bin/env python3
"""Same-scale 3D point clouds for zeta/RVM walks and two residual walks."""

from __future__ import annotations

import argparse
import hashlib
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from scipy.special import lambertw

from data_io import load_float64


TWOPI = 2.0 * np.pi


def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def lambert_levels(count: int) -> np.ndarray:
    ordinal = np.arange(1, count + 1, dtype=np.float64)
    argument = (ordinal - 11.0 / 8.0) / np.e
    tau = TWOPI * (ordinal - 11.0 / 8.0) / np.real(lambertw(argument, 0))
    tau = tau.astype(np.longdouble)
    target = ordinal.astype(np.longdouble) - np.longdouble(0.5)
    two_pi = np.longdouble(2.0) * np.longdouble(np.pi)
    for _ in range(5):
        value = (
            (tau / two_pi) * np.log(tau / (two_pi * np.e))
            + np.longdouble(7.0 / 8.0)
            - target
        )
        derivative = np.log(tau / two_pi) / two_pi
        tau -= value / derivative
    return tau.astype(np.float64)


def common_limits(first: np.ndarray, second: np.ndarray, padding: float = 0.06) -> tuple[float, float, float,
     float]:
    real_min = float(min(np.min(first.real), np.min(second.real)))
    real_max = float(max(np.max(first.real), np.max(second.real)))
    imag_min = float(min(np.min(first.imag), np.min(second.imag)))
    imag_max = float(max(np.max(first.imag), np.max(second.imag)))
    real_pad = padding * max(real_max - real_min, 1.0)
    imag_pad = padding * max(imag_max - imag_min, 1.0)
    return real_min - real_pad, real_max + real_pad, imag_min - imag_pad, imag_max + imag_pad
```

```
def plot_pair(
    first: np.ndarray,
    second: np.ndarray,
    first_title: str,
    second_title: str,
    suptitle: str,
    output: Path,
    sample: np.ndarray,
) -> tuple[float, float, float, float]:
    limits = common_limits(first, second)
    colors = np.linspace(0.0, 1.0, sample.size)
    x_axis = (sample + 1) / first.size
    fig = plt.figure(figsize=(12.0, 5.2), dpi=180)
    axes = [fig.add_subplot(1, 2, 1, projection="3d"), fig.add_subplot(1, 2, 2, projection="3d")]

    for axis, values, title in zip(axes, (first, second), (first_title, second_title)):
        axis.scatter(
            x_axis,
            values.real[sample],
            values.imag[sample],
            c=colors,
            cmap="viridis",
            s=2.3,
            alpha=0.72,
            linewidths=0,
            rasterized=True,
        )
        centroid = np.mean(values)
        axis.scatter(
            [0.5],
            [centroid.real],
            [centroid.imag],
            color="#c8681c",
            marker="X",
            s=55,
            depthshade=False,
        )
        axis.set_xlim(0.0, 1.0)
        axis.set_ylim(limits[0], limits[1])
        axis.set_zlim(limits[2], limits[3])
        axis.set_xlabel(r"ordinal␣fraction␣$n/N$", labelpad=7)
        axis.set_ylabel(r"$\Re$␣cumulative␣walk", labelpad=7)
        axis.set_zlabel(r"$\Im$␣cumulative␣walk", labelpad=7)
        axis.set_title(title, loc="left", pad=10)
        axis.view_init(elev=24, azim=-57)
        axis.grid(True, linewidth=0.35, alpha=0.5)

    fig.suptitle(suptitle, x=0.02, y=1.01, ha="left", fontsize=12.2)
    fig.tight_layout()
    output.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(output, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    return limits


def main() -> None:
    root = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, default=root / "data" / "g_hp.bin")
    parser.add_argument("--points", type=int, default=4500)
    args = parser.parse_args()

    gamma = load_float64(args.input)
    tau = lambert_levels(gamma.size)
```

```
    zeta_phase = np.exp(1j * gamma)
    rvm_phase = np.exp(1j * tau)
    epsilon = gamma - tau

    zeta_walk = np.cumsum(zeta_phase)
    rvm_walk = np.cumsum(rvm_phase)
    paired_walk = np.cumsum(zeta_phase - rvm_phase)
    intrinsic_walk = np.cumsum(np.exp(1j * epsilon) - 1.0)

    sample = np.unique(
        np.round(np.linspace(0, gamma.size - 1, min(args.points, gamma.size))).astype(int)
    )
    walk_output = root / "figures" / "zeta_rvm_walks_3d_same_scale.png"
    residual_output = root / "figures" / "residual_walks_3d_same_scale.png"
    walk_limits = plot_pair(
        zeta_walk,
        rvm_walk,
        "Zeta-zero␣exponential␣walk",
        "Lambert--RVM␣carrier␣walk",
        "Zeta␣and␣RVM␣cumulative␣phase␣clouds␣on␣identical␣axes",
        walk_output,
        sample,
    )
    residual_limits = plot_pair(
        paired_walk,
        intrinsic_walk,
        r"Paired␣phase␣residual␣$e^{i\gamma_k}-e^{i\tau_k}$",
        r"Intrinsic␣residual␣$e^{i(\gamma_k-\tau_k)}-1$",
        "Two␣residual␣cumulative␣clouds␣on␣identical␣axes",
        residual_output,
        sample,
    )

    rvm_residual = (
        (tau / TWOPI) * np.log(tau / (TWOPI * np.e))
        + 7.0 / 8.0
        - (np.arange(1, gamma.size + 1) - 0.5)
    )
    print("Same-scale␣3D␣phase-walk␣clouds")
    print(f"input␣=␣data/{args.input.name}")
    print(f"input␣sha256␣=␣{file_sha256(args.input)}")
    print(f"ordinates␣=␣{gamma.size}")
    print(f"sampled␣points␣per␣panel␣=␣{sample.size}")
    print(f"max␣|M(tau_k)-(k-1/2)|␣=␣{np.max(np.abs(rvm_residual)):.3e}")
    print(f"zeta␣walk␣centroid␣=␣{np.mean(zeta_walk).real:+.12f}␣{np.mean(zeta_walk).imag:+.12f}i")
    print(f"RVM␣walk␣centroid␣=␣{np.mean(rvm_walk).real:+.12f}␣{np.mean(rvm_walk).imag:+.12f}i")
    print(f"paired-residual␣walk␣centroid␣=␣{np.mean(paired_walk).real:+.12f}␣{np.mean(paired_walk).imag:+.12f}
         i")
    print(f"intrinsic-residual␣walk␣centroid␣=␣{np.mean(intrinsic_walk).real:+.12f}␣{np.mean(intrinsic_walk).
         imag:+.12f}i")
    print(
        "walk␣common␣limits␣(Re_min,␣Re_max,␣Im_min,␣Im_max)␣=␣"
        + "␣".join(f"{value:+.6f}" for value in walk_limits)
    )
    print(
        "residual␣common␣limits␣(Re_min,␣Re_max,␣Im_min,␣Im_max)␣=␣"
        + "␣".join(f"{value:+.6f}" for value in residual_limits)
    )
    print(f"figure␣=␣figures/{walk_output.name}")
    print(f"figure␣=␣figures/{residual_output.name}")


if __name__ == "__main__":
    main()
```

```
Same-scale 3D phase-walk clouds
input = data/g_hp.bin
input sha256 = 80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2
ordinates = 22491
sampled points per panel = 4500
max |M(tau_k)-(k-1/2)| = 3.638e-12
zeta walk centroid = +0.777204389314 +0.768179110409i
RVM walk centroid = +0.041475031748 +1.757470238988i
paired-residual walk centroid = +0.735729357566 -0.989291128579i
intrinsic-residual walk centroid = -422.852350549495 -0.256907673382i
walk common limits (Re_min, Re_max, Im_min, Im_max) = -2.488223 +4.021101 -2.482510 +3.937423
residual common limits (Re_min, Re_max, Im_min, Im_max) = -815.598452 +48.714749 -2.850584 +0.828066
figure = figures/zeta_rvm_walks_3d_same_scale.png
figure = figures/residual_walks_3d_same_scale.png
```

Appendix O. SHA-256 manifest

```
c7b955b1fb2bf28daf31de6ddb5412c0454d168f62de5c7511a3d0ec54f0f480 ./README.md
80c7bb512e210922397fd36b5bdfb97239cc0c5ee73dc1ec57a919ce41b367b2 ./data/g_hp.bin
2a3eb56f7c9e269c9291e8a31b96cc155c72e8d1307ca58d992c662fe8eb3e36 ./data/g_hp.npy
5d99a93b6d2df679714b6bc46b3cf16e20b3eb8e22566e6595e8582ac2502742 ./data/gammas.bin
5362c5c5e682167d956e460c17959209e900435f524acbc93d14ed41a43e601e ./data/gammas.npy
ae9d35415ee924504638d96e9778a2f63f20e9c1d2ae14ce3ecf35f775bce57a ./data/rvm_tau_2001052.bin
f8163985438l9d2adb9da2539aa0960933ca8167f8a5ba677ec1e387f62279c5 ./figures/carrier_saddle_validation.png
2a53ef410e5d6dfbae8e0a1d95f66002fa25c4d2ff6ec777a7202f444ddf6ffa ./figures/centroid_rate_validation.png
4925991bf6d207cf5e9ba52d50e2b0a4f82f518b32a85d39a9e723db338caded ./figures/connected_form_factor.png
47dc6f92257b5a6b39e31ce5731a134d274529cc80bef22faaceed3c29b1672b ./figures/core_theorem_compatibility.png
d1d9ed31e4d1f5fc1c84da7e94c2e59f2d5d974fca8af9e7853130430fdfc5a2 ./figures/prime_model_validation.png
a111d48fd0680aa9fe81eb6db93e2b67e4614a951cc2c9ed9b3ef3edb987d7d7 ./figures/residual_walks_3d_same_scale.png
7a7545cd81afc9b4e1e39c9779b1d100c779d4d25879891cb63d54bf867f591a ./figures/rvm_counterexample_validation.png
ff9b6b69fbd04c7bd1aa1d611b65f4e7afb8335d23c0121d5150cb4710bc616c ./figures/unfolded_residual_pair_law.png
0a035188c782fffd9aa0c8d5e0769ba62441e8b73cab50c7ec0bce45d3bbbed7 ./figures/zeta_rvm_walks_3d_same_scale.png
9887ebb7299835d1736a212e264293d24801b907cc351c149d0f675684230f7b ./montgomery_from_exponential_centroids_v1.
    tex
4899289463ad6039ef8410943976d79703eaf5a82ecad83401b0e4aca8dadc8f ./outputs/carrier_saddle_validation_output.
    txt
6c4620d865d2e8fb0eba79af39f265170d02442243f7c18e03633ccca02ad6d7 ./outputs/centroid_rate_validation_output.txt
67df6bc35502e5b673003e78835fa3430cdd5613da9fe13201357f1ae0fe7a3c ./outputs/connected_form_factor_output.txt
79360acf6f46de87521e5920b4333b0ebf70a8709ce01b1ac4f8e94885ddbd1c ./outputs/core_theorem_compatibility_output.
    txt
52d4321eb33d54edbf5ef7ffb81a610515095caf82779395c5b3d7d6a03d7275 ./outputs/export_binary_data_output.txt
e318e6239bc6dda3f1f168561af926a155fdb1d9171dc4560ddff2055640b808 ./outputs/generate_rvm_binary_output.txt
db515ddb1f8c9bf69bac99b0168e0af809169a0612c1c559f1307b8ab399b1c2 ./outputs/phase_walks_3d_output.txt
5dc38a486e718cb37a5fb2a427bade03dc9ebc7bd9a44db9852097c13650386b ./outputs/prime_model_validation_output.txt
85d029c3aea2cbc0edaa11168c37077f930b205b5847fc3a41641c5f3081da34 ./outputs/
    rvm_counterexample_validation_output.txt
b35e3152f4ed17ed5a83fa467364751f28058d4f004d072878ffc3f7cadee95d ./outputs/unfolded_residual_pair_law_output.
    txt
206141107330db8e8d131373724d7f12a1d79b8bd2a42b897aaf423f866a5e8c ./requirements.txt
f3c18d5c8520e9bec74983b24c63eed0b1147f4a5b780b4d752350578db994bb ./run_all.sh
205d53d8faa032b5626da8fcb939871b7bc0203b36d9abf8a013097625df1f76 ./scripts/analyze_unfolded_residual_pair_law.
    py
7d6820b0afbcf3319ddffd47c065caa38597f0ff4e942dba8f40179eebb2ea47 ./scripts/carrier_saddle_validation.py
9005069809fcf712de7ab26a5a9f0c4b2e7101fcd9afe1603ce8d14da3b18cfe ./scripts/centroid_rate_validation.py
adff0cf9d3b8f02ebc6a297b2a48207b7b529b692b638ec08e9d327754dbfc85 ./scripts/connected_form_factor.py
fce12df4ea6e82165be9ac6eeea164dd32fc2f1c4dd950fde149cfb80ae3510a ./scripts/data_io.py
402818513e44ba91a5964a6d587be0d21a0ce48df542869ad8bb7f3a8fbc05b7 ./scripts/export_binary_data.py
c5d0e229e106e4c86b11fd78542613169fa70ce3807bbdfa89088c64a0a74abc ./scripts/generate_rvm_binary.py
2ee483b68f22f4f50a373c7dc8ce668f5aabfde227e22c9e156d9084e31f8dce ./scripts/phase_walks_3d.py
cbafbbcb4b3748b747f7f387e4f2b3782adb44ca8211bbda0d15f472d982eb57 ./scripts/prime_model_validation.py
bd3cd9ac58f4a068f68b22660375a4d47ee9faf50986010f2aefa3dc3b2c8c44 ./scripts/rvm_counterexample_validation.py
ecdbe92c6787a6aadaf00922df80f35a389ebaf322fb647a9101d7e579a60b17 ./scripts/verify_core_theorems.py
```

```
bc789efdd6fb1fddddc60ef3c3aa297b1c6a934a02eda7a4c109548574916aaf ./source_archive/all_uploaded_material/4
    models.png
25eaf2395d386dd6ac108867a034e26584872b136923a653dc8a218370f2aded ./source_archive/all_uploaded_material/
    FFTcontinuous.png
cee7851572e908869e908cd20f2d1aeb523de5320a7e6637817675b172aed5b8 ./source_archive/all_uploaded_material/
    FFTzeta.png
01543da940a9704a1b8e950c04194a62427c4bda81c5d4b29d53b261dcb64fb2 ./source_archive/all_uploaded_material/Histo.
    png
86dca3b0cdfd7a50fcf516d170bc096a220a0f22a2edbd39de148461a3b615e9 ./source_archive/all_uploaded_material/
    PhaseDistribution(2).tex
a1824f9f779eb9163b66aecaf575ae6dc83413e9cdd7ddb5503d908027d3e06c ./source_archive/all_uploaded_material/
    SN_CN_Central.tex
51ca0ded08339db283daadc96f1abb258c784892f0c7e3a549c65ede620134fa ./source_archive/all_uploaded_material/
    SN_CN_Central_original.tex
c1f953bd7b86ddc1bdb8ffcaff570469c5217e72925ffccc9a0b2735cb26a2ce ./source_archive/all_uploaded_material/
    SNdensity(1).png
4539623739f9511b1fa21a6daa323cf9759559f71f6bd23ff303c578469f5b5e ./source_archive/all_uploaded_material/
    Zfases_FINAL(1).tex
aa44f952c0b5925282cadbfc17bb29d69355d2540d7e25fb1847855e013acca6 ./source_archive/all_uploaded_material/
    de1b4227-c9bd-47f7-9ccd-a10f650a9da6.zip
7951d57f2c6217c811ae04e9fffbd88b648dcd6734c58c593bd96725ec228e64 ./source_archive/all_uploaded_material/fases.
    png
cd582e6d5f2e223d0d7001df0e3e1590885fe937d9cfb67a30b646f51aad49ef ./source_archive/all_uploaded_material/
    formfactor.png
bdb07bb618e6b98481b3e3c26e35b9ba2b65a01db172d0caa6152eb989b33d96 ./source_archive/all_uploaded_material/
    formfactor_pack.zip
9bdd0f1c8d7f0825baeee7476583bf11a74035706881421e6c4ca2ed4689fb49 ./source_archive/all_uploaded_material/
    nradden(1).png
2d14aae818307054dd45d56e0d142d3a1d5ae04389164a60817b9853a654e9ac ./source_archive/all_uploaded_material/
    zeta_rvm_exponential_centroids_v14(1).tex
aa44f952c0b5925282cadbfc17bb29d69355d2540d7e25fb1847855e013acca6 ./source_archive/de1b4227-c9bd-47f7-9ccd-
    a10f650a9da6.zip
bdb07bb618e6b98481b3e3c26e35b9ba2b65a01db172d0caa6152eb989b33d96 ./source_archive/formfactor_pack.zip
532946ae26dc4c8241b8af9bb14f66897972f3712255823ebd47ecc6772bfed8 ./source_archive/formfactor_pack_extracted/
    SHA256SUMS.txt
d4c0191b82c54d103ba823be2517d5a1548e15e7686d802cb77b3c31d3e6160a ./source_archive/formfactor_pack_extracted/
    corrII.npz
58a8a3df6b6bf1ee5c32b059cf58f1c9980e61c6562b1da54b991b1514c47d11 ./source_archive/formfactor_pack_extracted/
    corrII.py
1d30d19d0c7d96e32f4eaed4cd7d9877c1be39302bfeae3ab4f83a377cde89fd ./source_archive/formfactor_pack_extracted/
    corrII_scale.py
6cc36905eaa62f78b504ad4b12cd91e1fab88567f78c4f245284ab87e5ca86a5 ./source_archive/formfactor_pack_extracted/
    dump.py
4cc4e57d2884309a05c0d2ae4b38986f8210f351b8ec79ae4274f480fac66beb ./source_archive/formfactor_pack_extracted/
    fig3.py
cd582e6d5f2e223d0d7001df0e3e1590885fe937d9cfb67a30b646f51aad49ef ./source_archive/formfactor_pack_extracted/
    formfactor.png
6c4c15259a130897e3ac5b6fb8047d92abd695327615b8f14542abc034dbf5f8 ./source_archive/formfactor_pack_extracted/
    formfactor_data.txt
2a3eb56f7c9e269c9291e8a31b96cc155c72e8d1307ca58d992c662fe8eb3e36 ./source_archive/formfactor_pack_extracted/
    g_hp.npy
a75f1e3bebe895fdc3aa1c1ff85bd540b150075d14918d2a02bdb31bea0006cc ./source_archive/formfactor_pack_extracted/
    output_completo.txt
cd582e6d5f2e223d0d7001df0e3e1590885fe937d9cfb67a30b646f51aad49ef ./source_archive/formfactor_supplied.png
```